FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research

Working Draft

28 April 2026

Contents

1 Overview

FEMaster is a finite-element solver for linear mechanical analyses. Models are described with an input-deck language in which commands start with an asterisk, command options are written as keywords, and numerical data follows on subsequent lines.

The manual is organized as a user reference. Command tables summarize the available DSL commands; the later chapters explain how model entities are connected, how collectors are activated in loadcases, and how analysis controls affect the solved equations.

1.1 Model layers

A model starts with nodes. Nodes define global coordinates and provide the reference points for all other model entities. Elements connect nodes into line, surface, or volume discretizations. Surfaces describe element sides used by pressure loads, distributed loads, ties, and surface couplings. Sets group nodes, elements, or surfaces so that loads, supports, sections, and constraints can refer to meaningful names instead of long ID lists.

Materials and sections give elements their physical meaning. An element by itself defines only topology and interpolation. A section assigns material, thickness, area, or profile data. Features such as point masses are additional model objects that contribute mass, inertia, or spring stiffness without being part of element connectivity.

Loads and supports are stored in named collectors. A loadcase decides which collectors are active. This allows one model to contain several analyses without repeating geometry, materials, sections, or load definitions.

1.2 Supported analysis types

Type	Purpose
`LINEARSTATIC`	Small-displacement static equilibrium with linear material behaviour.
`LINEARSTATICTOPO`	Static analysis with element-wise topology fields such as density, orientation, and penalization exponent.
`EIGENFREQ`	Natural-frequency analysis of the linear constrained system.
`LINEARBUCKLING`	Linear buckling analysis from a static preload and the resulting geometric stiffness.
`LINEARTRANSIENT`	Linear time integration with Newmark parameters, optional Rayleigh damping, and controlled result cadence.

1.3 Degree-of-freedom convention

Mechanical nodal vectors use the component order

`Ux, Uy, Uz, Rx, Ry, Rz`.

The first three components are translations and the last three are rotations. Not every element uses all six components. Solid elements usually contribute only translational displacement degrees of freedom. Beams, shells, constraints, connectors, and some features may introduce rotations.

1.4 Targets

Many commands accept a target that can be either a set name or a single numeric ID. For example, `*CLOAD` can target a node set or one node, `*DLOAD` can target a surface set or one surface, and `*VLOAD` can target an element set or one element. This makes small test decks compact while keeping large models readable through named sets.

2 Build and Command Line

FEMaster is built with CMake. A typical build produces an executable named `FEMaster` in the build output directory. The executable reads an input deck and writes a text result file. If no explicit output path is supplied, the result file uses the input base name with the extension `.res`.

2.1 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

A release build is the normal choice for analysis work. A debug build is useful for small diagnostic models, assertions, and development checks, but it is usually too slow for larger finite-element models.

2.2 Running an input deck

```
FEMaster model.inp
FEMaster model.inp --output model.res
FEMaster model.inp --ncpus 8
```

Argument	Meaning
`input_file`	Path to the input deck. If no `.inp` suffix is supplied, it is added automatically.
`-output FILE`	Explicit result file path. Without this option, a `.res` file is derived from the input name.
`-ncpus N`	Number of threads allowed for parallel assembly and compute sections.

2.3 DSL documentation mode

The executable can print DSL registration information for the installed version. This is useful when checking which commands, keywords, and variants a particular build accepts.

```
FEMaster --doc-list
FEMaster --doc-show CLOAD
FEMaster --doc-variants ELASTIC
```

Option	Meaning
`-doc-list`	Print the available DSL commands.
`-doc-show CMD`	Show details for one command.
`-doc-tokens CMD`	Show expected tokens for a command.
`-doc-variants CMD`	Show accepted data-line variants.
`-doc-search TEXT`	Search command names and descriptions.
`-doc-where-token TOKEN`	Find commands that use a token.
`-doc-all`	Print the full command overview.

3 DSL Basics

3.1 Commands

A FEMaster deck consists of commands and data lines. A command starts with an asterisk, followed by the command name and optional comma-separated keywords.

Syntax

```
*COMMAND, KEY=value, FLAG
data, data, data
```

Command names and keywords are normalized by the parser. Case is not significant, and spaces in command names are removed before lookup. A consistent uppercase style is still recommended for readability.

3.2 Scopes

Some commands open an active context. `*MATERIAL` activates a material; subsequent material subcommands such as `*ELASTIC`, `*DENSITY`, and `*THERMALEXPANSION` apply to it. `*LOADCASE` activates an analysis context; commands such as `*SUPPORTS`, `*LOADS`, `*SOLVER`, and `*CONSTRAINTMETHOD` configure that loadcase.

No explicit end command is required. A context ends when a following command no longer belongs to it or when the file ends.

3.3 Aliases, defaults, and flags

Many keywords have aliases. `NAME` and `NSET` can identify node sets; `MATERIAL` and `MAT` are equivalent in sections. Defaults reduce boilerplate: `*NODE` defaults to `NALL`, `*ELEMENT` defaults to `EALL`, and `*SOLVER` defaults to `DEVICE=CPU` and `METHOD=DIRECT`.

Flags are keywords without a required value. `GENERATE` is the most common example. Explicit false-like values such as `NO`, `OFF`, `FALSE`, or `0` are interpreted as inactive.

3.4 Component order

Mechanical nodal vectors use `Ux, Uy, Uz, Rx, Ry, Rz`. The first three values are translations. The last three values are rotations. Loads, supports, velocities, accelerations, displacements, and many internal reductions use this convention.

3.5 Targets

A target can often be a set name or a single numeric ID. If a matching set name exists, the set is used. Otherwise FEMaster attempts to interpret the token as one ID.

4 Command Reference

This chapter gives a compact overview of the DSL commands. The following chapters describe the commands in more detail.

4.1 Geometry and topology

Command	Purpose
*NODE	Define nodes and coordinates.
*ELEMENT	Define elements by type and connectivity.
*SURFACE	Define element sides or element-set sides as surfaces.
*NSET	Define node sets.
*ELSET	Define element sets.
*SFSET	Define surface sets.

4.2 Properties

Command	Purpose
*MATERIAL	Create or activate a material.
*ELASTIC	Assign elastic material behaviour.
*DENSITY	Assign mass density.
*THERMALEXPANSION	Assign thermal expansion coefficient.
*SOLIDSECTION	Assign material and optional orientation to solid elements.
*SHELLSECTION	Assign material, thickness, and optional orientation to shell elements.
*BEAMSECTION	Assign material, profile, and local axis data to beam elements.
*TRUSSSECTION	Assign material and area to truss elements.
*PROFILE	Define beam profile constants.

4.3 Features, fields, loads, and constraints

Command	Purpose
*POINTMASS	Add concentrated mass, rotational inertia, and optional spring stiffness to node sets.
*FIELD	Define node, element, or integration-point data fields.
*ORIENTATION	Define a rectangular or cylindrical local coordinate system.
*SUPPORT	Add support entries to a support collector.

Command	Purpose
*CLOAD	Add concentrated nodal loads to a load collector.
*DLOAD	Add vector surface tractions to a load collector.
*PLOAD	Add pressure loads to a load collector.
*VLOAD	Add volume loads to a load collector.
*TLOAD	Add thermal loads from a temperature field.
*INERTIALOAD	Add rigid-body inertial loads.
*AMPLITUDE	Define a time function for transient load scaling.
*COUPLING	Define kinematic or structural couplings.
*TIE	Define tie constraints between slave nodes and master geometry.
*CONNECTOR	Define connector constraints between two node sets.
*RBM	Generate rigid-body-motion suppression equations.

4.4 Loadcase controls

Command	Purpose
*LOADCASE	Start an analysis block.
*SUPPORTS	Activate support collectors.
*LOADS	Activate load collectors.
*SOLVER	Select device and linear solution method.
*CONSTRAINTMETHOD	Select Nullspace or Lagrange constraint handling.
*INERTIARELIEF	Enable inertia relief for free static models.
*REBALANCELOADS	Balance residual resultant force and moment.
*REQUESTSTIFFNESS	Request stiffness matrix output.
*REQUESTSTGEOM	Request geometric stiffness matrix output.
*NUMEIGENVALUES	Set the number of eigenpairs.
*SIGMA	Set the buckling eigenvalue shift.
*TOPODENSITY	Select the topology density field.
*TOPOORIENT	Select the topology orientation field.
*TOPOEXPONENT	Set the topology penalization exponent.
*TIME	Set transient time range and step size.
*NEWMARK	Set Newmark integration parameters.
*DAMPING	Set Rayleigh damping.
*WRITEEVERY	Set transient output cadence.
*INITIALVELOCITY	Set transient initial velocity from a field.
*CONSTRAINTSUMMARY	Request generated-constraint diagnostics.
*OVERVIEW	Print a model overview.

5 Nodes

Nodes are the geometric reference points of a FEMaster model. Each node has an ID and three global coordinates. Elements, loads, supports, constraints, and features refer to nodes either directly or through node sets.

Syntax

```
*NODE, NSET=<optional-set>
<id>, <x>, <y>, <z>
```

`NSET` is optional. If omitted, nodes are added to `NALL`. Missing coordinate values are interpreted as zero, but production decks should normally write all three coordinates explicitly.

A node may have up to six mechanical degrees of freedom, `Ux, Uy, Uz, Rx, Ry, Rz`. Which of these are active depends on the connected elements and model objects. Solid elements usually need only translations. Shells, beams, point-mass springs, connectors, and constraints may require rotations.

5.1 Node IDs

Node IDs must be non-negative integers. The ID `0` is valid and can be used like any other node ID. Negative node IDs are not valid user-facing node identifiers in input decks because node IDs are used directly as row indices in nodal fields, connectivity, loads, supports, and result output.

FEMaster does not require a particular numbering strategy. Clear numbering is still useful for debugging, set construction, and result inspection. IDs should be unique and stable. If the same node ID is defined multiple times, the deck no longer gives a clear single coordinate for that node.

5.2 Coordinate meaning

Node coordinates are always global coordinates. Local coordinate systems do not change node positions; they only change how vector components are interpreted by commands such as `*SUPPORT`, `*CLOAD`, `*DLOAD`, `*VLOAD`, `*SOLIDSECTION`, `*SHELLSECTION`, and `*CONNECTOR`. This distinction matters: geometry is global, while loads, supports, material axes, and connector directions may be local.

5.3 Active degrees of freedom

FEMaster builds the active degree-of-freedom set from the model objects that reference each node. A node connected only to solid elements normally contributes three translational degrees of freedom. A shell or beam node contributes translations and rotations. A connector, coupling, support, or point-mass spring can also make rotational degrees of freedom relevant.

The presence of a node alone does not imply six active unknowns. The active set is determined when

the model is assembled. This keeps purely solid models smaller while still allowing mixed solid-shell-beam models to share the same nodal component convention.

6 Elements

Elements connect nodes and define the interpolation used over a line, surface, or volume region. They are the fundamental discretization objects of the finite-element model: once a mesh has been read, the selected element types determine how nodal degrees of freedom are converted into strains, curvatures, stresses, internal forces, mass contributions, geometric stiffness terms, and recoverable result fields.

An element alone does not define a complete physical model. A usable model also needs sections, materials, coordinate systems, loads, supports, and analysis controls. The element type defines the kinematic assumption and the numerical integration domain; the section and material then define how that kinematic state becomes stiffness, mass, stress, or section force.

Syntax

```
*ELEMENT, TYPE=<type>, ELSET=<optional-set>
<element-id>, <node-1>, <node-2>, ...
```

`TYPE` is required. `ELSET` is optional and defaults to `EALL`. Element IDs should be non-negative and unique because they are referenced by element sets, sections, loads, result fields, diagnostics, and postprocessing operations. Connectivity order matters: it defines the local reference orientation, face numbering, integration point layout, and, for structural elements, the local axes used for section behaviour.

Element quality has a direct influence on solution quality. Poorly shaped elements, excessive distortion, highly warped shell surfaces, inverted Jacobians, and inappropriate element choices can significantly reduce accuracy even if the solver converges successfully. A coarse model made from a suitable element family is often more useful than a fine model made from an element type that does not match the physical idealization.

6.1 Registered element types

Type	Nodes	Family
`C3D4`	4	Linear tetrahedral solid.
`C3D5`	5	Pyramid input mapped to a degenerated hexahedral solid.
`C3D6`	6	Linear wedge solid.
`C3D8`	8	Linear hexahedral solid.
`C3D10`	10	Quadratic tetrahedral solid.
`C3D13`	13	Quadratic pyramid solid.
`C3D15`	15	Quadratic wedge solid.
`C3D20`	20	Quadratic hexahedral solid with full higher-order integration.
`C3D20R`	20	Quadratic hexahedral solid with reduced stiffness integration and higher-order mass integration.
`S3`	3	Three-node triangular shell.
`S4`	4	Four-node quadrilateral shell.
`MITC4`	4	Four-node quadrilateral shell with MITC-style tied shear interpolation.

Type	Nodes	Family
S6	6	Six-node quadratic triangular shell.
S8	8	Eight-node quadratic quadrilateral shell.
QSPT	4	Specialized four-node quadrilateral shear-panel type.
B33	2 or 3	Two-node 3D beam element with optional orientation node.
T33	2	Two-node truss element.

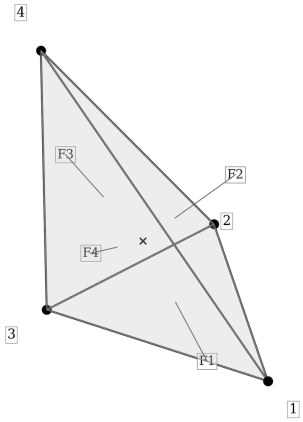
6.2 Solid element family

Solid elements describe three-dimensional continua. They use translational nodal degrees of freedom and require *SOLIDSECTION; the section assigns material data and, where needed, a material orientation. Solid elements are appropriate when the stress state is genuinely three-dimensional, when through-thickness stresses matter, or when local geometry, joints, load introduction, contact, or nonlinear local deformation cannot be represented by a shell or line idealization.

The main cost of solid modelling is the number of degrees of freedom required to resolve a body. A solid plate, for example, usually needs several elements through thickness before bending and transverse stress fields become credible. Low-order solids are inexpensive and robust, but they may become overly stiff in bending-dominated states. Higher-order solids improve stress gradients and curved geometry representation, but they are more sensitive to distortion and use larger element matrices.

For most engineering models, hexahedral elements are preferable when a high-quality structured or swept mesh is available. Tetrahedral elements are valuable for complex geometry and automated meshing, especially in their quadratic form. Wedges and pyramids are often transition elements: they are useful when mesh topology changes, but they should not be used casually in regions where they dominate the stiffness or stress result.

6.3 C3D4: 4-node tetrahedral solid



Element	C3D4
Type	solid
Nodes	4
Faces	4
Edges	6
DOF / node	3
Total DOF	12
Shape functions	linear tetrahedral Lagrange
Integration points	1

Figure 6.1: C3D4 element topology and integration scheme.

Syntax

```
*ELEMENT, TYPE=C3D4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

C3D4 is the simplest three-dimensional continuum element in FEMaster. It uses four corner nodes, a tetrahedral reference domain, and linear displacement interpolation. Because the interpolation is linear, the strain field inside a single element is constant. The element is therefore easy to integrate and very robust from a meshing perspective, but it has limited ability to represent bending, curvature, or stress gradients without substantial mesh refinement.

The strongest reason to use C3D4 is geometric robustness. Tetrahedra can fill complicated CAD volumes, imported parts, transition regions, and local details with relatively little manual meshing effort. For early model checks, contact-region filler meshes, stiffness estimates, or coarse load-path studies, this can be more important than high stress accuracy. A model built from C3D4 elements is often a useful first step because it can reveal gross constraint errors, load direction mistakes, disconnected parts, and order-of-magnitude stiffness behaviour.

The weakness of C3D4 is accuracy per degree of freedom. In bending-dominated problems, linear tetrahedra tend to be too stiff unless the mesh is very fine. Since each element carries only a constant strain state, stress contours can look patchy and stress gradients are represented mostly by element-to-element jumps. This is especially visible near holes, notches, fillets, and load introduction regions. If the goal is engineering stress recovery rather than only global stiffness, C3D10 is usually a better tetrahedral choice.

When using C3D4, keep the mesh quality conservative. Avoid sliver elements, inverted tetrahedra, and regions where many badly shaped elements align in the same stress path. Refinement should be guided by displacement convergence and by the stability of integrated quantities such as reactions and strain energy, not only by local peak stress values. The element is robust, but it should not be mistaken for a high-accuracy stress element.

6.4 C3D5: 5-node pyramid input

Syntax

```
*ELEMENT, TYPE=C3D5, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <apex>
```

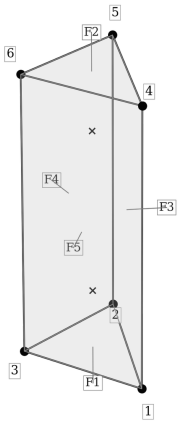
C3D5 accepts a five-node pyramid-style connectivity. In FEMaster this topology is treated primarily as an input convenience: the pyramid is mapped internally to a degenerated hexahedral representation by repeating the apex node in the upper hexahedral positions. This allows mixed meshes containing pyramid transition cells to be imported without requiring the user to remesh every transition region by hand.

The element is useful when a mesh transitions between a quadrilateral or hexahedral region and a tetrahedral region. Such transitions occur frequently in automatic mesh generation, especially around complex local features attached to otherwise block-like geometry. In that role, C3D5 is a compatibility element. It helps preserve mesh topology and lets the surrounding solid elements carry the main structural response.

It should not be interpreted as a dedicated high-quality pyramid formulation. Degenerated mappings can produce less predictable numerical behaviour than regular tetrahedra, wedges, or hexahedra. The quality of the base quadrilateral, the height of the pyramid, and the position of the apex strongly influence the Jacobian and therefore the stiffness contribution. Flat, skewed, or highly stretched pyramids can become poor transition cells even if the surrounding mesh looks acceptable.

Use C3D5 deliberately and preferably in small numbers. It is reasonable in local transition zones where it improves mesh compatibility, but it should not dominate a stress-critical region. If many pyramid elements appear in the primary load path, inspect the mesh and compare against a refined or alternative discretization. For stress-sensitive work, a clean tetrahedral or hexahedral mesh is normally more predictable than a large population of degenerated pyramid cells.

6.5 C3D6: 6-node wedge solid



Element	C3D6
Type	solid
Nodes	6
Faces	5
Edges	9
DOF / node	3
Total DOF	18
Shape functions	linear wedge / prism
Integration points	2

Figure 6.2: C3D6 wedge element.

Syntax

```
*ELEMENT, TYPE=C3D6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

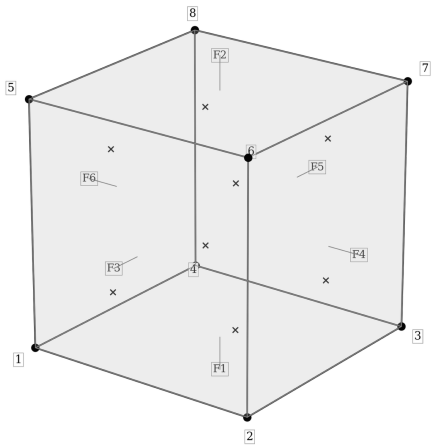
C3D6 is a linear wedge, or prism, solid. It combines triangular topology in one parametric direction with quadrilateral side faces in the other directions. The element is useful for swept meshes, extruded triangular meshes, layered solids, and transition regions where a purely hexahedral topology is not available but a tetrahedral mesh would not align well with the geometry.

The usual strength of a wedge is directional structure. If a part has a natural extrusion direction, such as a layer stack, a swept rib, a boundary-layer-like region, or a prismatic transition, wedge elements can align the mesh with that direction more cleanly than tetrahedra. This can improve through-thickness consistency and make section-like stress interpretation easier. Wedges can also reduce the number of elements needed in regions where triangular surface meshing is unavoidable but the volume is still essentially swept.

As a low-order element, C3D6 still has limited bending performance. If only one or two linear wedge elements are used through thickness, bending-dominated states can be too stiff and stress recovery can be coarse. A wedge mesh should therefore be refined in the direction where gradients are expected, especially through thickness and near load introduction. The element is not a substitute for a higher-order formulation when curvature or stress gradients are central to the analysis.

Mesh quality is particularly important for wedges because their triangular and quadrilateral faces can distort in different ways. Collapsed triangular faces, strongly skewed side faces, and elements with a poor height-to-base relationship can produce unreliable Jacobians. A good wedge mesh should have consistent layer direction, limited twisting between the two triangular faces, and smooth size transitions. When used in that role, C3D6 is a practical compromise between fully structured hexahedral meshes and fully unstructured tetrahedral meshes.

6.6 C3D8: 8-node hexahedral solid



Element	C3D8
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange
Integration points	8

Figure 6.3: C3D8 hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

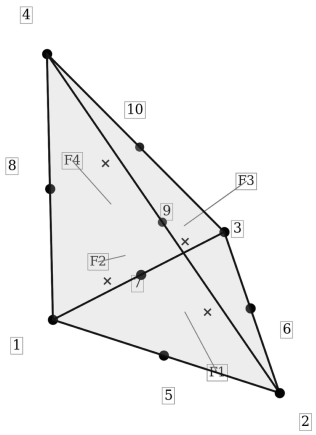
C3D8 is the standard low-order hexahedral solid element. It uses eight corner nodes and a hexahedral reference domain. In a well-shaped mesh, the element is often more efficient than a comparable low-order tetrahedral discretization because it aligns naturally with block-like geometry, swept volumes, layered structures, and orthotropic material directions.

The element is a good default for regular solid meshes. It performs well when the mesh follows the structural load path and when element directions are meaningful: for example in plates modelled with solids, rectangular blocks, extruded parts, bonded layers, spars, ribs, and regions where material orientation should follow a local coordinate system. Hexahedral elements also make it easier to control the number of elements through thickness, which is important for bending and local stress gradients.

The main limitation is still the linear displacement interpolation. A coarse C3D8 mesh can be too stiff in bending, and one element through thickness is rarely enough for credible stress results in a bending-dominated solid model. Curved boundaries are represented only by straight element edges, so geometric approximation error can also become significant unless the mesh is refined or a higher-order hexahedral element is used.

Element distortion matters. Warped faces, skewed edges, inverted Jacobians, and extreme aspect ratios reduce the reliability of stiffness and stress recovery. A distorted hexahedron can be worse than a cleaner tetrahedral mesh. Use C3D8 when the mesh can remain structured and reasonably regular; for complex geometry where a high-quality hex mesh is not practical, a quadratic tetrahedral mesh may produce more predictable stress behaviour.

6.7 C3D10: 10-node quadratic tetrahedral solid



Element	C3D10
Type	solid
Nodes	10
Faces	4
Edges	6
DOF / node	3
Total DOF	30
Shape functions	quadratic tetrahedral Lagrange
Integration points	4

Figure 6.4: C3D10 quadratic tetrahedral element.

Syntax

```
*ELEMENT, TYPE=C3D10, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>
```

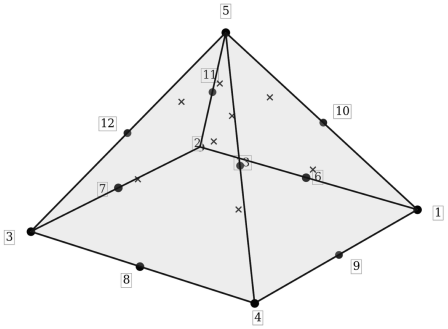
C3D10 is the quadratic tetrahedral solid element. It adds midside nodes to the four-node tetrahedral topology and uses quadratic interpolation over the tetrahedral reference domain. Compared with C3D4, it can represent curved displacement fields, bending deformation, and stress gradients much more effectively.

This element is usually the preferred tetrahedral choice for engineering stress analysis. It keeps the meshing flexibility of tetrahedra while avoiding many of the stiffness and stress-recovery limitations of linear tetrahedra. For complex castings, imported CAD geometry, organic shapes, local fillets, holes, and automatically generated meshes, C3D10 often gives a better balance between meshing robustness and numerical quality than a very fine C3D4 mesh.

The midside nodes are important. If they lie on curved geometric edges and surfaces, the element can represent curved boundaries more accurately than a linear tetrahedron. If they are badly placed, however, the mapping can become distorted. A quadratic tetrahedron with poor midside-node placement can have a poor Jacobian even when the corner-node tetrahedron looks acceptable. Mesh generation and geometry projection therefore matter more than they do for C3D4.

The cost per element is higher, but fewer elements are often required for a comparable displacement or stress result. In stress-critical regions, convergence should still be checked by refinement. Quadratic interpolation improves the element, but it does not remove the need for reasonable aspect ratios, clean transition sizes, and careful interpretation of peak stresses near singularities, sharp corners, and idealized point loads.

6.8 C3D13: 13-node quadratic pyramid solid



Element	C3D13
Type	solid
Nodes	13
Faces	-
Edges	8
DOF / node	3
Total DOF	39
Shape functions	quadratic pyramid
Integration points	8

Figure 6.5: C3D13 quadratic pyramid element.

Syntax

```
*ELEMENT, TYPE=C3D13, ELSET=<set>
<id>, <n1>, ..., <n13>
```

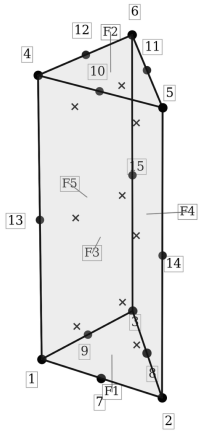
C3D13 is a higher-order pyramid transition element. It is intended for mixed meshes where a hexahedral or quadrilateral topology must connect to a tetrahedral topology. The additional midside nodes give the element more flexibility than a linear pyramid-style input and allow it to participate in higher-order solid meshes.

Pyramid elements are topologically useful but numerically delicate. Their reference-to-physical mapping is more complicated than the mapping of a tetrahedron, wedge, or hexahedron. The element has a quadrilateral base and triangular side faces, which makes it attractive as a transition cell, but also means that element quality depends on both the base shape and the apex position. Poor pyramid geometry can create strong Jacobian variation and unreliable stress recovery.

Use C3D13 as a transition element rather than as the primary building block of a model. It is reasonable around mesh interfaces where a structured hexahedral region joins an automatically meshed tetrahedral region. It is less attractive in large homogeneous blocks, in bending-critical regions, or in zones where the pyramid elements would carry a significant part of the stiffness response.

When a model contains C3D13 elements, inspect where they are located. If they appear only in local transition bands, they are usually acceptable. If they form long chains through the load path or cluster near stress-critical features, compare the result against a cleaner mesh. The element exists to preserve compatibility in mixed meshes, not to replace well-shaped hexahedral or tetrahedral discretizations.

6.9 C3D15: 15-node quadratic wedge solid



Element	C3D15
Type	solid
Nodes	15
Faces	5
Edges	9
DOF / node	3
Total DOF	45
Shape functions	quadratic wedge / prism
Integration points	9

Figure 6.6: C3D15 quadratic wedge element.

Syntax

```
*ELEMENT, TYPE=C3D15, ELSET=<set>
<id>, <n1>, ..., <n15>
```

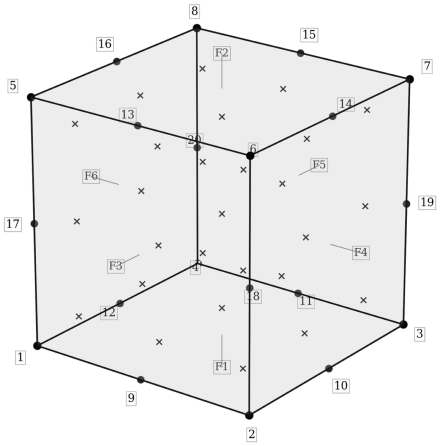
C3D15 is the quadratic counterpart to C3D6. It uses a wedge topology with midside nodes and higher-order interpolation. The element is useful when the geometry has a swept or layered character but the response requires smoother displacement and stress fields than a linear wedge can provide.

In practice, C3D15 is attractive for extruded triangular meshes, boundary-layer-type regions, curved swept solids, and transition zones where wedge topology is natural. It can represent bending and through-thickness gradients better than C3D6, and it can fit curved edges more accurately when midside nodes are projected correctly to the geometry.

The element is still sensitive to mesh quality. A wedge can distort through triangular face shape, through side-face warping, or through inconsistent movement of midside nodes. Because the interpolation is higher order, a bad geometric mapping can have a stronger effect than it would in a linear wedge. The two triangular faces should remain consistently oriented, and the side faces should not twist excessively.

Use C3D15 when wedge topology is needed and the result quality justifies the additional cost. It is a good candidate for refined swept regions, but not a cure for poor mesh topology. If the geometry can be meshed cleanly with quadratic hexahedra, C3D20 may be more efficient for smooth fields. If the geometry is highly unstructured, C3D10 may be easier to generate robustly.

6.10 C3D20: 20-node quadratic hexahedral solid



Element	C3D20
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	27

Figure 6.7: C3D20 quadratic hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D20, ELSET=<set>
<id>, <n1>, ..., <n20>
```

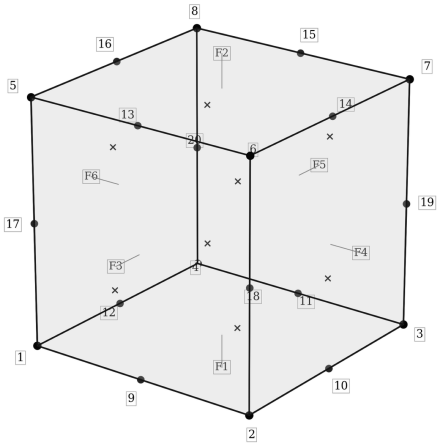
C3D20 is the full higher-order hexahedral solid element. It adds midside nodes to the standard hexahedral topology and uses quadratic interpolation with higher-order integration. In a high-quality structured mesh, this is one of the most accurate continuum element choices available in FEMaster.

The element is well suited to smooth stress fields, bending-dominated solids, curved geometry, and problems where stress gradients matter. Compared with C3D8, it can represent curvature and deformation modes with fewer elements, especially when the physical solution is smooth and the mesh follows the geometry. It is a strong choice for block-like parts, swept volumes, thick plates, and local refinement regions where a structured quadratic mesh can be maintained.

The extra nodes and integration points increase cost. Each element contributes larger matrices and requires more memory and assembly work than a linear hexahedron. This cost is often worthwhile when it reduces the number of elements required for convergence, but it can be wasteful in regions where the solution is nearly uniform or where the mesh is too distorted to benefit from higher-order interpolation.

Mesh quality is the deciding factor. A well-shaped C3D20 mesh can produce excellent results, but a distorted higher-order hexahedron can produce misleading stresses. Midside nodes should follow intended straight or curved edges consistently, and the element should not be warped into shapes that create poor Jacobians. Use this element when the geometry and meshing process can support it; otherwise, a cleaner lower-order or tetrahedral discretization may be more reliable.

6.11 C3D20R: 20-node reduced-integration hexahedral solid



Element	C3D20R
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	8

Figure 6.8: C3D20R reduced-integration quadratic hexahedral element.

Syntax

```
*ELEMENT, TYPE=C3D20R, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20R uses the same 20-node quadratic hexahedral topology as C3D20, but applies reduced stiffness integration. Reduced integration lowers the cost of stiffness assembly and can improve behaviour in situations where a fully integrated element is prone to excessive stiffness, such as bending-dominated or locking-sensitive states.

This element is useful when a high-quality quadratic hexahedral mesh is available and the model benefits from reduced integration. It can provide good bending behaviour with less assembly cost than the fully integrated element. The mass and stiffness integration choices should be understood as part of the element formulation rather than as a simple performance switch: they change the numerical character of the element.

The main risk is the appearance of nonphysical deformation patterns. Reduced integration can introduce zero-energy or hourglass-like modes if the formulation, constraints, or mesh quality do not control them sufficiently. Such modes may not be obvious from solver convergence alone. They often show up as unexpected displacement shapes, unrealistic local distortion, poor reaction balance, or stress fields that do not match the expected load path.

Use C3D20R with engineering checks. Inspect deformed shapes, reaction forces, strain energy distribution, and stress smoothness, especially for coarse or distorted meshes. If the model is sensitive, compare against C3D20 or a refined mesh. Reduced integration is powerful when used deliberately, but it should not be selected only because it is cheaper.

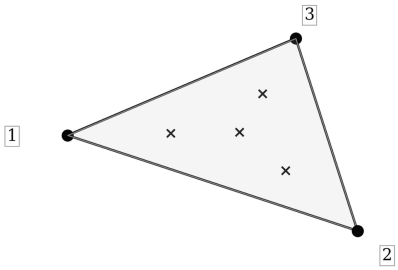
6.12 Shell element family

Shell elements describe thin-walled structures using translational and rotational degrees of freedom. They require `*SHELLSECTION`, which supplies material, thickness, optional orientation, transverse shear stiffness, and offsets. A shell model replaces a thin three-dimensional body with a two-dimensional mid-surface plus section behaviour. This makes shells much more efficient than solid meshes for plates, skins, panels, webs, pressure-vessel walls, and thin-walled mechanical components.

The shell idealization is appropriate when the thickness is small compared with the in-plane dimensions and when through-thickness stress details are not the main modelling target. Shells recover membrane behaviour, bending curvature, transverse shear behaviour, and section resultants. They are not intended to resolve local three-dimensional stress concentrations through thickness unless the model is enriched with local solid detail.

Shell quality depends on surface mesh quality, normal consistency, aspect ratio, warping, and thickness assumptions. A shell element can be geometrically thin but numerically poor if it is badly warped or if the local normals flip across the mesh. For thin bending-dominated structures, the choice between triangular, quadrilateral, low-order, and higher-order shells can matter as much as mesh density.

6.13 S3: 3-node triangular shell



Element	S3
Type	shell
Nodes	3
Faces	-
Edges	3
DOF / node	6
Total DOF	18
Shape functions	linear triangular shell
Integration points	4

Figure 6.9: `S3` triangular shell element.

Syntax

```
*ELEMENT, TYPE=S3, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

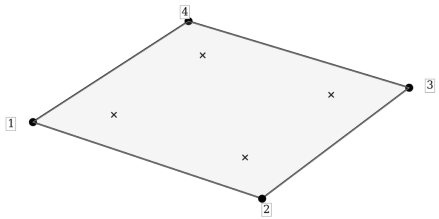
`S3` is a three-node triangular shell element. It is geometrically flexible and can discretize complex surfaces, imported shell meshes, transition regions, and local details where quadrilateral meshing is difficult. Because any polygonal surface can be decomposed into triangles, `S3` is often the easiest shell element to generate automatically.

The element should be viewed as a robust and convenient shell, not as the most accurate bending element. Triangular shells can be stiffer than good quadrilateral shells in bending-dominated situations, and they may require substantial refinement to produce smooth curvature and stress-resultant fields. This is especially true on broad regular panels where a structured quadrilateral mesh could align with the primary bending and membrane directions.

`S3` is useful in irregular regions where mesh topology is more important than high-order accuracy. It can also be used to close gaps between quadrilateral regions, to represent complex cutouts, or to support imported shell meshes from external preprocessors. When using it in a stress-critical region, refine the mesh enough that local element orientation does not dominate the result.

Prefer quadrilateral shell elements in smooth, regular regions when possible. Use `S3` where triangular topology solves a real meshing problem. A mixed shell mesh can be perfectly reasonable if triangles are kept in transition zones and quadrilaterals carry the main panel behaviour.

6.14 S4: 4-node quadrilateral shell



Element	S4
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	bilinear quadrilateral shell
Integration points	4

Figure 6.10: S4 quadrilateral shell element.

Syntax

```
*ELEMENT, TYPE=S4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

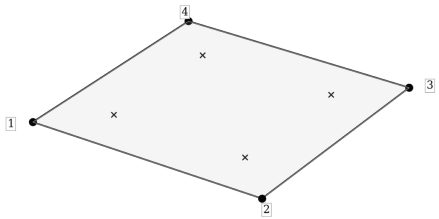
S4 is the standard four-node quadrilateral shell element. It is a practical default for many thin-walled structures when a reasonably regular quadrilateral mesh is available. Quadrilateral topology aligns naturally with plates, panels, webs, skins, and other structural surfaces where two in-plane directions are meaningful.

Compared with triangular shells, quadrilateral shells usually provide smoother membrane and bending fields for the same number of elements, provided the mesh quality is good. A structured S4 mesh can represent panel behaviour efficiently and gives intuitive local element directions for interpreting section resultants. It is therefore a good first choice for regular shell regions.

The limitations are connected to low-order interpolation and mesh distortion. A coarse S4 mesh can be too stiff in thin bending problems, and strongly warped or skewed quadrilaterals can reduce both stiffness accuracy and stress recovery quality. If transverse shear locking is suspected, MITC4 may be more appropriate. If the surface is smooth and higher-order meshing is available, S8 can offer better convergence.

Use S4 with attention to aspect ratio, warping, and normal direction. A clean mesh of simple quadrilateral shells is often more valuable than a complicated higher-order mesh with poor geometry. For large panel models, refine gradually and compare reactions, deflections, and section resultants rather than relying only on local peak stress values.

6.15 MITC4: quadrilateral shell with tied shear interpolation



Element	MITC4
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	bilinear MITC quadrilateral shell
Integration points	4

Figure 6.11: MITC4 shell element.

Syntax

```
*ELEMENT, TYPE=MITC4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

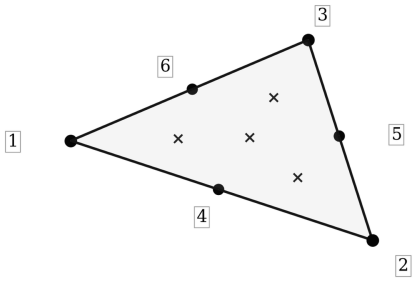
MITC4 has the same four-node quadrilateral topology as S4, but modifies the transverse shear interpolation. FEMaster evaluates shear-related quantities at tied locations and interpolates them back into the element. The purpose is to reduce shear locking, which can otherwise make low-order shell elements artificially stiff in thin bending-dominated structures.

This element is often the safer low-order quadrilateral shell when the structure is thin, the mesh is relatively coarse, or bending response is more important than membrane response. It is useful for plates, skins, webs, stiffened panels, and similar structures where a standard low-order shear interpolation may overestimate stiffness.

MITC4 does not remove the need for a good mesh. Severe warping, poor aspect ratios, inconsistent normals, and abrupt size changes can still damage the solution. It also remains a low-order element, so smooth curvature and stress gradients still require refinement. The tied shear interpolation addresses a specific numerical weakness; it does not make the element insensitive to modelling choices.

When comparing S4 and MITC4, check global displacement, reaction forces, and section-resultant patterns. If S4 appears too stiff in a thin-shell bending problem, MITC4 is usually worth trying. If both elements give similar behaviour on a refined mesh, the simpler quadrilateral shell may be sufficient.

6.16 S6: 6-node quadratic triangular shell



Element	S6
Type	shell
Nodes	6
Faces	-
Edges	3
DOF / node	6
Total DOF	36
Shape functions	quadratic triangular shell
Integration points	4

Figure 6.12: S6 quadratic triangular shell element.

Syntax

```
*ELEMENT, TYPE=S6, ELSET=<set>
<id>, <n1>, ..., <n6>
```

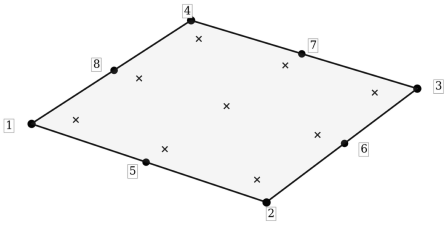
S6 is the quadratic triangular shell element. It extends the triangular shell topology with midside nodes and higher-order interpolation. The result is a shell element that keeps triangular meshing flexibility while improving curved-surface representation and field smoothness compared with S3.

This element is useful when triangular topology is unavoidable but the response requires better bending or stress-resultant quality than a low-order triangle can provide. It is a good candidate for curved shell surfaces, irregular imported meshes, and local features where quadrilateral meshing is not practical. The midside nodes allow the element to follow curved boundaries and surfaces more closely when the mesh generator places them correctly.

The cost and geometric sensitivity are higher than for S3. Midside-node placement must be consistent with the intended surface; otherwise, the element can become geometrically distorted even if the corner triangle looks acceptable. A poor higher-order triangle can be less reliable than a cleaner low-order element.

Use S6 when the surface geometry or solution field justifies quadratic interpolation. It is not a reason to ignore mesh quality or convergence checks. For broad regular shell regions, a high-quality quadrilateral mesh may still be more efficient. For irregular curved regions, S6 can be a useful compromise between geometric flexibility and accuracy.

6.17 S8: 8-node quadratic quadrilateral shell



Element	S8
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	quadratic serendipity shell
Integration points	9

Figure 6.13: S8 quadratic quadrilateral shell element.

Syntax

```
*ELEMENT, TYPE=S8, ELSET=<set>
<id>, <n1>, ..., <n8>
```

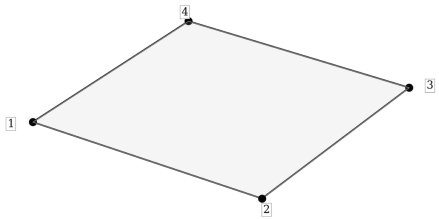
S8 is the higher-order quadrilateral shell element. It uses midside nodes and quadratic interpolation, making it suitable for smooth shell fields, curved boundaries, and bending-dominated surfaces where a low-order mesh would require many elements.

In a high-quality mesh, S8 can represent curvature, displacement gradients, and section-resultant variation more efficiently than S4. It is particularly useful on smooth panels, curved shells, pressure-vessel surfaces, and refined local regions where stress recovery matters. The element can reduce mesh density requirements when the geometry and solution are smooth enough to benefit from higher-order interpolation.

The disadvantages are cost and sensitivity to element shape. Warped or distorted higher-order quadrilaterals can produce unreliable results, and incorrectly placed midside nodes can damage the geometric mapping. A higher-order element is not automatically better if the mesh quality is poor.

Use S8 when the shell surface is smooth and the mesh generator can produce clean curved quadrilaterals. For rough imported meshes, abrupt topology changes, or strongly irregular panels, a simpler refined low-order mesh may be easier to validate. As with solid higher-order elements, convergence should be checked using global and integrated quantities as well as local stress measures.

6.18 QSPT: quadrilateral shear-panel type



Element	QSPT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	3
Total DOF	12
Shape functions	quadrilateral shear-panel formulation
Integration points	-

Figure 6.14: QSPT shear-panel element.

Syntax

```
*ELEMENT, TYPE=QSPT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

QSPT is a specialized four-node quadrilateral panel element nach Bäß, the inventor of the element developed at SLA. It is intended for shear-flow-oriented structural idealizations and should not be treated as a general shell replacement. Its purpose is to represent panel-like shear transfer in simplified structural models where shear flow or effective panel stiffness is the main quantity of interest.

This kind of element is useful in aircraft-style thin-walled structures, webs, simplified stiffened panels, or preliminary models where the detailed shell kinematics are less important than the panel shear path. It can provide a compact representation of shear transfer without requiring a full shell formulation for every panel feature.

The limitation is generality. QSPT is not meant to capture the same bending, rotational, and shell-resultant behaviour as S4, MITC4, or S8. If the model depends on bending stiffness, drilling behaviour, local shell curvature, or detailed shell stresses, a standard shell element is the more appropriate choice.

Use QSPT only when the engineering idealization matches the element behaviour. It is valuable when a panel is intended to carry shear in a simplified structural model. It is risky when used simply because it has four nodes and resembles a quadrilateral shell in the mesh.

6.19 Beam and truss element family

Line elements reduce a three-dimensional structural member to a one-dimensional representation with section properties. They are efficient for slender members because the cross-section is described by a section definition rather than by a solid or shell mesh. This makes them useful for frames, rods, stiffeners, trusses, spars, and reinforcement members.

The price of this efficiency is the modelling assumption. Local three-dimensional stress states, connection details, bolt holes, weld geometry, local buckling modes, and cross-section deformation are not resolved unless the model adds local detail elsewhere. Line elements are best used when member forces and global stiffness are the design quantities.

6.20 B33: 3D beam element

Element	B33
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	6
Total DOF	12
Shape functions	linear 2-node beam
Integration points	2

Figure 6.15: B33 beam element.

Syntax

```
*ELEMENT, TYPE=B33, ELSET=<set>
<id>, <n1>, <n2>, <orientation-node>
```

B33 is a two-node 3D beam element with six degrees of freedom per node. It represents axial deformation, bending, torsion, and section orientation through a beam-section description. The optional third connectivity entry is an orientation node used to define the local beam-axis system.

The element is appropriate for slender structural members where section forces are the natural engineering output. Frames, stiffeners, spars, ribs, support structures, and idealized reinforcements can often be modelled with far fewer degrees of freedom than a comparable shell or solid model. Because the section properties are defined explicitly, the model can represent axial stiffness, bending inertias, torsional stiffness, principal-axis orientation, and offsets without meshing the full cross-section.

The main limitation is the beam assumption. B33 does not resolve local stress concentrations, cross-section warping beyond the supported formulation, detailed joint flexibility, bolt patterns, weld geometry, or local load introduction effects. If the member is short and thick, if the joint detail dominates the response, or if local shell or solid stresses are required, a beam idealization may be too coarse.

The local axis definition must be robust. The orientation direction should not be parallel to the beam axis, and adjacent beams should use consistent orientations when section-force interpretation matters. After building a beam model, inspect local axes, reactions, and internal forces before relying on stress-like quantities derived from the section.

6.21 T33: 2-node truss

Element	T33
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	3
Total DOF	6
Shape functions	linear 2-node truss
Integration points	1

Figure 6.16: T33 truss element.

Syntax

```
*ELEMENT, TYPE=T33, ELSET=<set>
<id>, <n1>, <n2>
```

T33 is a two-node truss element. It carries axial force along its line direction and uses translational degrees of freedom only. The element requires *TRUSSSECTION, which supplies material and cross-sectional area.

The element is appropriate for members intended to carry tension or compression without bending moment transfer. Pin-jointed structures, rods, links, braces, idealized cables, and simplified support members are typical use cases. Because the formulation is simple, truss models can be very efficient and easy to interpret: the important output is usually axial force, axial strain, and the corresponding stress based on section area.

The limitations are deliberate. T33 has no bending stiffness, no torsional stiffness, and no rotational continuity. It cannot represent frame action, fixed-end moments, eccentric connections, or transverse stiffness except through the geometry of the truss network. If a physical member can transfer moment or has important bending response, use a beam or shell model instead.

Truss models are sensitive to structural mechanism behaviour. A collection of axial-only members must be geometrically stable under the applied supports and loads. If the model contains mechanisms, the solver may report singularity or produce unrealistic displacements. Before using truss results for design, check that the assumed pin-jointed idealization is actually consistent with the structure being represented.

6.22 Choosing an element

Element selection should follow the physical idealization before it follows meshing convenience. Use solids when the stress state is three-dimensional or when local geometry and through-thickness behaviour matter. Use shells when the structure is thin-walled and section resultants are meaningful. Use beams when members are slender and section forces are the primary design quantities. Use trusses only when axial force is the intended behaviour.

For solid meshes, prefer hexahedra when a high-quality structured mesh is available, use quadratic tetrahedra for serious stress work on complex geometry, and treat pyramids mainly as transition elements. For shell meshes, prefer quadrilaterals in regular regions, use triangles where topology requires them, and consider `MITC4` when thin-shell bending appears too stiff. For line models, verify that the connection assumptions match the physical structure.

No element choice removes the need for validation. Check reactions, displacement shapes, load paths, mesh sensitivity, and stress convergence. A good element used outside its assumptions can be less reliable than a simpler element used in a model that matches the physics.

7 Surfaces

Surfaces describe loadable or connectable element sides. They are used for pressure loads, distributed surface loads, tie constraints, and surface couplings. A surface is a selection of element sides; it is not a material or section property.

7.1 Surface from one element side

Syntax

```
*SURFACE, NAME=<surface-set>
<surface-id>, <element-id>, <side>
```

The side can be written as a number or as a token such as `S1` or `S2`. For shells, `SPOS` and `SNEG` select the positive or negative shell side.

7.2 Surface from an element set

Syntax

```
*SURFACE, NAME=<surface-set>, TYPE=ELEMENT
<element-set-or-id>, <side>
```

If the target is an element set, FEMaster creates surfaces for the selected side of all elements in the set. Otherwise the target is interpreted as a single element ID.

7.3 Orientation and sign

Surface loads depend on the chosen side and its normal direction. If pressure or traction direction is unexpected, check the selected side, element orientation, and shell side. For shells, positive and negative sides share the same geometric midsurface but have opposite normals.

8 Sets

Sets are named collections of model IDs. They are one of the most important readability tools in a FEMaster input deck because most higher-level commands should target named sets instead of repeating long lists of node, element, or surface IDs. A set gives a modelling meaning to otherwise anonymous mesh numbers.

The mesh IDs themselves are usually not stable engineering concepts. They can change when a part is remeshed, when an external preprocessor exports a new deck, or when local refinement is added. A set name such as `FIXED_EDGE`, `WING_SKIN`, `LOAD_PATCH`, or `BOLT_SURFACES` is much more meaningful than a raw list of IDs. The name records why those entities are grouped, not only which numbers happened to exist in one version of the mesh.

Sets are also useful because FEMaster commands often operate on groups. Supports, loads, sections, fields, surface interactions, couplings, and diagnostics all become easier to audit when their targets are named explicitly. A deck that assigns a shell section to `PANEL_TOP_SKIN` or a support to `ROOT_CLAMP_NODES` can be reviewed without constantly cross-checking individual IDs against the mesh.

8.1 *NSET: node sets

Syntax

```
*NSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

A node set groups node IDs. If `GENERATE` is present, FEMaster interprets the data line as a range with start ID, end ID, and increment. Without `GENERATE`, the following data lines are read as explicit node IDs. Generated sets are compact and convenient for regular ID ranges, while explicit sets are better for arbitrary selections imported from a preprocessor.

Node sets are used wherever the model needs to refer to points in the discretization. Supports and concentrated loads commonly target node sets. Couplings, rigid-body-style constraints, connector definitions, point masses, diagnostic output, and result extraction workflows also use them. A good node set name should describe the physical or modelling role of the nodes, such as `FIXED_ROOT`, `LOAD_NODES`, `COUPLING_SLAVES`, or `MASS_POINTS`.

Node sets should be chosen with the intended degree-of-freedom behaviour in mind. A set applied to solid-only nodes usually affects translational degrees of freedom. A set applied to shell or beam nodes may involve rotations as well. If a support or load appears to have no effect, verify that the targeted node set contains the expected nodes and that those nodes actually own the degrees of freedom referenced by the command.

It is normal for one node to belong to several node sets. For example, a node can be part of a geometric set, a load-introduction set, and a diagnostic output set at the same time. This is not a problem by itself. The important question is whether the commands using those sets create consistent boundary conditions and loads. Overlapping sets become dangerous only when they accidentally apply contradictory constraints or duplicate loads.

Example

```
*NSET, NAME=FIXED_ROOT, GENERATE
1, 41, 1
```

Example

```
*NSET, NAME=LOAD_PATCH
105, 106, 112, 113, 119, 120
```

8.2 *ELSET: element sets

Syntax

```
*ELSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

An element set groups element IDs. As with node sets, `GENERATE` creates a compact range, while explicit data lines list individual element IDs. Element sets are central to physical model definition because sections are assigned to element sets. Without meaningful element sets, material and section assignment becomes difficult to review.

Element sets are used by solid, shell, beam, and truss sections, volume loads, inertial loads, topology fields, RBM constraints, and many postprocessing or diagnostic workflows. They should normally reflect physical regions or modelling roles: `SOLID_CORE`, `SHELL_SKIN`, `SPAR_BEAMS`, `TRUSS_BRACES`, `TOPO_DOMAIN`, or `GRAVITY_ELEMENTS`. Names like these make it clear why the elements are grouped and which later commands are expected to reference them.

One element may belong to several element sets. This is useful when the same elements need to be referenced for different reasons. A shell panel might belong to `PANEL_A`, `ALL_SHELLS`, and `AERO_LOAD_REGION`. A topology-design domain might also belong to a material section set. This kind of overlap is expected as long as the commands using the sets are compatible.

The main risk with element sets is unintended multiple physical assignment. If the same element is assigned to two different sections that are both meant to define its physical stiffness, the deck becomes ambiguous from a modelling perspective. Even if the parser accepts the input order, the model is difficult to audit. Keep physical section sets clear and non-overlapping unless there is a deliberate reason and the command semantics support it.

Generated element sets are convenient for simple meshes, but they can be fragile after remeshing. If element IDs change, a generated range may still exist but refer to the wrong region. For production decks, prefer sets exported from the preprocessor by named geometric selections when possible. The model should be organized around physical regions, not around accidental ID ranges.

Example

```
*ELSET, NAME=SOLID_CORE, GENERATE
1, 2400, 1
```

Example

```
*ELSET, NAME=SPAR_BEAMS
5101, 5102, 5103, 5104
```

8.3 *SFSET: surface sets

Syntax

```
*SFSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

A surface set groups surface IDs. Surface IDs are created by surface definitions and represent element faces, shell sides, or other surface-like model entities depending on the surrounding input. A surface set is therefore one level above the raw element topology: it names the boundary or interface region that later commands should use.

Surface sets are typically used by pressure loads, distributed loads, ties, contact-like workflows, and surface couplings. Commands such as `*PLOAD`, `*DLOAD`, `*TIE`, and coupling definitions become much clearer when they target names like `PRESSURE_SIDE`, `MASTER_SURFACE`, `SLAVE_SURFACE`, `BOLT_CONTACT`, or `LOAD_INTRODUCTION_SURFACE`.

The orientation of a surface matters. A surface is not only a collection of IDs; it also carries a geometric meaning through the underlying face orientation and normal direction. For pressure loads and interface constraints, the selected side of an element can change the physical interpretation. If a pressure has the wrong sign or a tie behaves unexpectedly, inspect both the surface set membership and the underlying surface orientation.

Surface sets can overlap, but overlapping surfaces should be reviewed carefully. A surface might reasonably belong to both a broad diagnostic set and a specific load set. It is less reasonable for the same surface region to accidentally receive two unrelated pressure definitions or to act as both sides of an interface constraint. Use narrow, descriptive names for operational surface sets so that accidental reuse is easier to spot.

As with element sets, generated surface ranges are compact but not inherently meaningful. They are useful for controlled meshes and hand-written examples. In larger workflows, named geometric selections exported from preprocessing are usually safer because they preserve the modelling intent when IDs change.

Example

```
*SFSET, NAME=PRESSURE_FACE, GENERATE
1001, 1040, 1
```

Example

```
*SFSET, NAME=TIE_SLAVE_SURFACE
2201, 2204, 2210, 2212
```

8.4 Generated and explicit sets

The `GENERATE` option is a compact way to define regular ID ranges. It is well suited to small examples, controlled test decks, and meshes whose numbering scheme is intentionally structured. A generated set is easy to read when the range corresponds to a physical region, such as all nodes along a simple edge or all elements in a block.

Explicit sets are more verbose, but they are often safer for arbitrary selections. If a preprocessor exports a named group of nodes, elements, or surfaces, the IDs may not form a simple range. In that case, an explicit list preserves exactly what was selected. Explicit sets are also useful for small hand-picked groups such as coupling reference nodes, load introduction nodes, or selected diagnostic elements.

The main disadvantage of both forms is that they still depend on numeric IDs. If the mesh is regenerated, the set content must be regenerated or checked. A generated range is especially easy to misuse after remeshing because it may still look plausible. Treat set definitions as part of the mesh data, not as permanent engineering truth independent of the mesh.

8.5 Naming conventions

Set names should describe function and physical meaning rather than only geometry. A name like `NODES_1` or `SET_A` provides almost no review value. A name like `FIXED_ROOT_NODES`, `PRESSURE_UPPER_SKIN`, `BOLT_HOLE_SURFACES`, or `SPAR_CAP_ELEMENTS` tells the reader what the set is for and why later commands reference it.

Names should also be specific enough to avoid accidental reuse. If a model has several load patches, `LOAD` is too broad. Use names such as `LOAD_PATCH_FRONT`, `LOAD_PATCH_REAR`, or `ACTUATOR_LOAD_NODES`. If a set is used only for output or diagnostics, include that in the name, for example `TIP_OUTPUT_NODES`. This prevents diagnostic sets from being mistaken for physical boundary-condition sets.

Consistency matters more than any one naming scheme. Choose a convention that distinguishes node sets, element sets, and surface sets clearly. Some decks use suffixes such as `_NODES`, `_ELEMENTS`, and `_SURFACES`; others rely on command context. Suffixes are verbose but helpful in large models because they make accidental cross-use easier to see.

8.6 Checking set usage

Set definitions should be reviewed whenever the mesh or preprocessing workflow changes. The solver can only operate on the IDs it receives; it cannot know whether a set still represents the physical region its name implies. A remeshed model may preserve a set name while changing its membership, or it may preserve numeric ranges while changing their geometric meaning.

Useful checks include verifying that every physical element region is covered by the intended section set, that every support and load set contains the expected nodes or surfaces, and that no operational set is empty. Empty sets are especially dangerous because they can make a command appear present in the deck while having no physical effect. Very small or unexpectedly large sets should also be treated as suspicious until confirmed.

For complex decks, it is often helpful to keep broad organizational sets separate from operational sets. A broad set such as `ALL_SHELLS` can be useful for output or global checks, while a narrow set such as `ROOT_CLAMP_NODES` should be used for a specific support. This separation makes the input deck easier to audit and reduces the risk of applying a command to more of the model than intended.

9 Sections and Profiles

Sections connect element sets to the physical data that the element topology alone does not contain. An element defines interpolation, degrees of freedom, and numerical integration; a section tells FEMaster which material, thickness, area, profile constants, or local orientation should be used for a particular element set. Without a matching section, elements may exist geometrically in the model, but they cannot contribute the intended stiffness, mass, stress recovery, or section-force output.

The separation between elements and sections is deliberate. A mesh can be grouped into named element sets, and each set can then receive the section definition that matches its physical role. This allows the same element type to represent different parts, materials, thicknesses, or idealizations without changing the mesh connectivity. It also makes the input deck easier to audit: element commands describe topology, while section commands describe physical interpretation.

Section definitions are normally applied to element sets through `ELSET`. The referenced set should contain only elements compatible with the section type. A solid section is meaningful for solid elements, a shell section for shell elements, a beam section for beam elements, and a truss section for truss elements. Mixing unrelated element families in the same section set is usually a modelling error even if the parser can read the command.

9.1 *SOLIDSECTION: solid sections

Syntax

```
*SOLIDSECTION, ELSET=<elset>, MATERIAL=<material>, ORIENTATION=<orientation>
```

A solid section assigns material behaviour to a set of three-dimensional continuum elements. It is used with the solid element family, such as `C3D4`, `C3D8`, `C3D10`, `C3D20`, and related transition elements. The section does not define a thickness or area because those quantities are already represented by the volume mesh. The physical volume, element shape, and integration rules provide the geometric contribution; the section supplies the constitutive interpretation.

`MATERIAL` is the central parameter. It references a material definition that provides elastic stiffness, density, or other material data used by the selected analysis type. The material assigned through the solid section affects stiffness assembly, mass assembly where density is available, stress recovery, and any loadcase that depends on material response.

`ORIENTATION` is optional. For isotropic materials it often has no practical effect because the material response is direction-independent. For orthotropic, anisotropic, or otherwise direction-dependent material models, however, orientation is part of the physical model. It defines how the material axes are placed relative to the element or global coordinate system. A wrong orientation can produce a model that is numerically stable but physically incorrect, especially in composite-like solids, layered inserts, printed structures, or directional materials.

Solid sections are usually simple, but they are easy to misuse in large models. The most common problem is an incomplete element-set assignment: some elements exist in the mesh but are not part of any solid section set, or are assigned to the wrong material. Another common issue is accidental reuse of a generic

set name when different regions should receive different materials. For serious models, inspect the section coverage and verify that each solid region has exactly the intended material and orientation.

Example

```
*SOLIDSECTION, ELSET=BLOCK, MATERIAL=STEEL
```

Example

```
*SOLIDSECTION, ELSET=ORTHO_CORE, MATERIAL=CORE_MAT, ORIENTATION=CORE_AXES
```

9.2 *SHELLSECTION: shell sections

Syntax

```
*SHELLSECTION, ELSET=<elset>, MATERIAL=<material>, THICKNESS=<t>, ORIENTATION=<orientation>
```

A shell section assigns material and thickness data to a set of shell elements. Shell elements represent a thin structure through a two-dimensional surface mesh, so the section carries information that would otherwise be present in a full three-dimensional solid mesh. The most important of these quantities is `THICKNESS`. It affects membrane stiffness, bending stiffness, transverse shear behaviour, mass, stress recovery, and shell resultants.

Thickness is not just a display property. In a shell formulation, membrane stiffness generally scales with thickness, while bending stiffness is much more sensitive because it depends on the thickness cubed in classical plate behaviour. A small thickness error can therefore have a large effect on bending response. The thickness value must be expressed in the same length unit as the node coordinates and the rest of the model.

`MATERIAL` references the material used by the shell section. For isotropic shells, the material and thickness together define the main stiffness behaviour. For orthotropic materials or ABD-type material descriptions, the shell orientation becomes part of the physical definition. It controls how material axes, laminate axes, or directional stiffness terms are aligned with the shell surface.

`ORIENTATION` is optional but important whenever the material is direction-dependent. It should be chosen so that the local material direction follows the intended engineering direction, such as panel length, fibre direction, stiffener direction, or manufacturing direction. The shell element itself also has a local surface orientation through its connectivity and normal direction. If shell normals are inconsistent or the material orientation is wrong, section resultants and stresses can be difficult to interpret even when the solver produces a displacement field.

Shell sections should be defined at the level of physical regions. If a panel has different thickness zones, each zone should normally receive its own element set and shell section. If a model uses offsets or more advanced section behaviour elsewhere in the input language, those settings should be kept consistent with the geometric idealization of the midsurface. A shell surface placed at a physical midplane behaves differently from a shell surface placed on an outer skin unless the section definition accounts for that modelling choice.

Example

```
*SHELLSECTION, ELSET=SKIN_TOP, MATERIAL=AL2024, THICKNESS=1.6
```

Example

```
*SHELLSECTION, ELSET=PANEL_45, MATERIAL=ORTHO_LAMINA, THICKNESS=0.8, ORIENTATION=PANEL_AXES
```

9.3 Beam profiles

Syntax

```
*PROFILE, NAME=<name>
<A>, <Iy>, <Iz>, <J>, <Iyz>, <y0>, <z0>, ...
```

A beam profile stores the cross-section constants used by beam sections. Beam elements reduce a three-dimensional member to a line, so the shape of the cross-section must be represented by section properties rather than by mesh geometry. The profile is the place where those properties are collected and named.

The most important values are the cross-sectional area A, the bending inertias I_y and I_z, the torsional constant J, and the product inertia I_{yz}. Area controls axial stiffness and mass contribution when density is available. The bending inertias control bending stiffness about the local section axes. The torsional constant controls twist stiffness. The product inertia is relevant when the chosen local axes are not principal axes of the section.

The coordinates y_0 and z_0, where used, describe section reference offsets in the local beam-section coordinate system. Offsets matter when the beam line does not pass through the intended section reference point. They can change coupling between axial, bending, and torsional behaviour and are especially important for idealized stiffeners, eccentric reinforcements, and members attached to shell surfaces.

The profile definition is intentionally independent from the beam element set. This allows a single profile to be reused by multiple beam sections, possibly with different materials or orientations. It also makes it possible to audit cross-section data separately from mesh topology. In larger decks, profile names should be descriptive enough to distinguish units and physical meaning; a profile named only `P1` is easy to confuse once several member types exist.

Profile constants must be in the unit system of the model. If length is measured in millimetres, area is in square millimetres and second moments of area are in fourth powers of millimetres. Unit mistakes in beam profiles are common because the exponents differ between area, bending inertia, and torsion. A section can appear connected and stable while still being wrong by orders of magnitude.

Example

```
*PROFILE, NAME=RECT_20_40
800.0, 106666.7, 26666.7, 90000.0, 0.0, 0.0, 0.0
```

9.4 *BEAMSECTION: beam sections

Syntax

```
*BEAMSECTION, ELSET=<elset>, MATERIAL=<material>, PROFILE=<profile>
<nx>, <ny>, <nz>
```

A beam section assigns a material and profile to a set of beam elements. The element connectivity defines the beam line, while the beam section defines how the line behaves as a physical member. The section references a *PROFILE for cross-section constants and a material for constitutive behaviour. Together they define axial stiffness, bending stiffness, torsional stiffness, mass contribution where applicable, and section-force recovery.

The direction line following the command defines a local section direction. It is used to construct the beam's local coordinate system and therefore determines which profile inertia acts about which physical direction. This direction must not be parallel to the beam axis. If it is nearly parallel, the local axis construction becomes ill-conditioned and the resulting section orientation may be unstable or unexpected.

Beam orientation is a modelling decision, not a cosmetic detail. A rectangular or otherwise non-axisymmetric profile has different bending stiffness about its local section axes. Rotating the section by 90 degrees can produce a completely different structural response. For open, offset, or asymmetric profiles, the orientation also affects how section forces and moments should be interpreted.

Beam sections are most reliable when the beam element set contains members with a consistent intended orientation. If a single set contains beams running in many directions, one shared orientation vector may not describe all members well. In that case, split the beams into separate sets or define orientations carefully so that the local axes match the engineering model. After building a beam model, inspect local axes and section forces before treating beam stresses as design quantities.

Example

```
*BEAMSECTION, ELSET=FRAME_LONGERONS, MATERIAL=AL2024, PROFILE=RECT_20_40
0.0, 0.0, 1.0
```

9.5 *TRUSSSECTION: truss sections

Syntax

```
*TRUSSSECTION, ELSET=<elset>, MATERIAL=<material>, AREA=<A>
```

A truss section assigns material and cross-sectional area to truss elements. Truss elements carry axial force only, so the section is correspondingly simple. The material provides the axial constitutive response, and `AREA` scales axial stiffness, mass contribution where density is available, and stress recovery.

The simplicity is useful, but it is also the main modelling constraint. A truss section does not define bending inertia, torsional stiffness, offsets, or rotational behaviour. If a physical member transfers moment, resists bending, or has important connection eccentricity, a truss section cannot represent that behaviour. In those cases a beam, shell, or solid idealization is more appropriate.

Area must be chosen consistently with the model unit system and with the intended structural idealization. For a rod-like member, it may correspond to the actual cross-sectional area. For an idealized brace, link, or equivalent stiffness member, it may represent an effective area chosen to match axial stiffness. In either case, the interpretation should be clear because truss stresses are based directly on axial force divided by area.

Truss sections should be assigned to element sets that are intended to behave as pin-jointed axial members. The surrounding support and connection model must provide stability. A collection of truss elements can easily form a mechanism if the geometry or constraints do not prevent rigid-body or hinge-like motion. When debugging truss models, check global stability, reaction balance, and axial force signs before focusing on local stresses.

Example

```
*TRUSSSECTION, ELSET=BRACES, MATERIAL=STEEL, AREA=25.0
```

9.6 Choosing and checking sections

A good section assignment should answer three questions: which elements receive the section, what physical data are assigned to them, and how local directions are interpreted. If any of these are unclear, the model is difficult to review and easy to misuse. Use meaningful element-set names, material names, profile names, and orientation names so the input deck remains readable after the mesh grows.

For solid sections, check material coverage and orientation for directional materials. For shell sections, check thickness, normals, and material axes. For beam sections, check profile units and local section directions. For truss sections, check that the member is really intended to carry axial force only. These checks are often faster than solving the model and can catch the most expensive modelling mistakes before assembly begins.

Sections should be reviewed whenever the mesh changes. A remeshed part may keep the same visual geometry but lose element-set membership, split a region into new sets, or change local element orientation. A section definition that was correct for an old mesh can silently become incomplete after preprocessing. Treat section coverage as part of model validation, not as a one-time setup step.

10 Features

Features are additional model objects that are not defined through element connectivity. They add concentrated properties or auxiliary behaviour to the model.

10.1 Point Mass

Syntax

```
*POINTMASS, NSET=<node-set>
<m>, <Ixx>, <Iyy>, <Izz>, <Kx>, <Ky>, <Kz>, <Krx>, <Kry>, <Krz>
```

`*POINTMASS` assigns concentrated mass, rotational inertia, and optional spring stiffness to the nodes of a node set. It is useful for equipment masses, simplified component masses, sensors, local inertias, or concentrated replacement springs.

The data line contains four groups: scalar mass, three rotational inertias, three translational spring stiffnesses, and three rotational spring stiffnesses. Missing values are interpreted as zero.

Point masses can contribute to mass and inertia. They are therefore important for eigenfrequency analysis, transient analysis, inertial loads, and inertia relief. In a purely static loadcase without inertia relief, mass does not automatically mean weight; gravity or acceleration must be introduced through loads.

`*INERTIALOAD` and `*INERTIARELIEF` use the option `CONSIDER_POINT_MASSES` to decide whether point-mass features are included in the mass and inertia balance.

11 Material Models

Material definitions provide density, thermal expansion, and elastic constitutive behaviour. Sections reference materials by name; elements receive their actual stiffness and mass only after their section has been assigned. A material can therefore be defined once and reused by several shell, solid, beam, or truss sections.

11.1 Material context: *MATERIAL

Syntax

```
*MATERIAL, NAME=<material>
  *ELASTIC, TYPE=<model>
  ...
  *DENSITY
  ...
  *THERMALEXPANSION
  ...
```

*MATERIAL activates or creates a named material entry. The material name is supplied through MATERIAL or the alias NAME. Subcommands such as *ELASTIC, *DENSITY, and *THERMALEXPANSION apply to the currently active material.

11.2 Density: *DENSITY

Syntax

```
*DENSITY
<rho>
```

*DENSITY sets the mass density ρ. Density is used when FEMaster builds mass matrices, lumped masses from continuum or structural elements, inertia loads, inertia relief mass properties, and dynamic equations. In a consistent unit system, ρ must match the length, force, and time units used in the deck. For example, if length is in meters and force in Newtons, density is normally given in kg/m^3.

For a continuum element, the mass contribution is based on

$$M_e = \int_{\Omega_e} \rho N^T N \, d\Omega$$

or on the corresponding lumped form used by the selected analysis. For shells and beams, the section thickness or section area converts density into mass per area or mass per length.

11.3 Thermal expansion: *THERMALEXPANSION

Syntax

```
*THERMALEXPANSION
<alpha>
```

*THERMALEXPANSION sets the scalar thermal expansion coefficient α. Thermal loads use this coefficient together with the temperature difference $T - T_0$. For isotropic expansion, the thermal strain vector is

$$\varepsilon_{\text{th}} = \alpha(T - T_0) \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}^T .$$

If a material has no thermal expansion coefficient, a thermal load can still be present in the deck, but it has no meaningful elastic thermal strain contribution for that material.

11.4 Isotropic elasticity

Syntax

```
*ELASTIC, TYPE=ISOTROPIC
<E>, <nu>
```

Accepted type values are ISO and ISOTROPIC. The data line contains Young's modulus E and Poisson's ratio ν. The shear modulus is not entered independently; it is derived as

$$G = \frac{E}{2(1+\nu)}.$$

For three-dimensional stress states, FEMaster uses the isotropic stiffness matrix

$$D = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & (1-2\nu)/2 & 0 & 0 \\ 0 & 0 & 0 & 0 & (1-2\nu)/2 & 0 \\ 0 & 0 & 0 & 0 & 0 & (1-2\nu)/2 \end{bmatrix}.$$

For two-dimensional plane-stress-style element formulations, the in-plane matrix is

$$D_{2D} = \frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}.$$

11.5 Generalised isotropic elasticity

Syntax

```
*ELASTIC, TYPE=GENISO
<E>, <nu>, <G>
```

Accepted type values are `GENERALISEDISOTROPIC`, `GENERALISED_ISOTROPIC`, and `GENISO`. This model keeps the normal isotropic stiffness terms from E and ν, but uses the supplied G as an independent shear stiffness. In 3D, the normal block is identical to the isotropic material, while the shear diagonal entries are

$$D_{44} = D_{55} = D_{66} = G.$$

In the 2D matrix, the shear entry is also set directly:

$$D_{2D,33} = G.$$

This material is intentionally not a physically general isotropic law when $G \neq E/(2(1+\nu))$. It breaks the isotropic relationship between Young's modulus, Poisson's ratio, and shear modulus. That can be useful deliberately, especially for shell studies where the user wants to tune, reduce, or effectively ignore shear stiffness while keeping isotropic membrane and bending terms. It should not be used as a physical isotropic material unless the supplied G satisfies the isotropic relation.

11.6 Orthotropic elasticity

Syntax

```
*ELASTIC, TYPE=ORTHOTROPIC
<E1>, <E2>, <E3>, <G23>, <G13>, <G12>, <nu23>, <nu13>, <nu12>
```

Accepted type values are `ORTHO` and `ORTHOTROPIC`. Orthotropic elasticity defines three material axes with separate Young's moduli E_1, E_2, E_3, shear moduli G_{23}, G_{13}, G_{12}, and Poisson ratios. It is appropriate for laminates, wood-like materials, unidirectional composites, and any material whose stiffness depends on direction.

The 3D compliance matrix is assembled as

$$S = \begin{bmatrix} 1/E_1 & -\nu_{21}/E_2 & -\nu_{31}/E_3 & 0 & 0 & 0 \\ -\nu_{12}/E_1 & 1/E_2 & -\nu_{32}/E_3 & 0 & 0 & 0 \\ -\nu_{13}/E_1 & -\nu_{23}/E_2 & 1/E_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/G_{23} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/G_{13} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/G_{12} \end{bmatrix}, \qquad D = S^{-1}.$$

Reciprocal Poisson ratios are derived from modulus ratios. In particular, FEMaster derives

$$\nu_{31} = \nu_{13}\frac{E_3}{E_1}.$$

The input column is named `NU13`, but the material construction uses the reciprocal value needed by the compliance matrix. Orthotropic data should be checked carefully because inconsistent modulus and Poisson-ratio combinations can produce a non-positive or ill-conditioned stiffness matrix.

11.7 ABD elasticity

Syntax

```
*ELASTIC, TYPE=ABD
<abd11>, <abd12>, ..., <abd66>,
<s11>, <s12>, <s21>, <s22>
```

`*ELASTIC, TYPE=ABD` reads 40 values. The first 36 values are a 6×6 ABD matrix in row-major order. The last 4 values are a 2×2 transverse shear matrix in row-major order. The values may be distributed over up to eight input lines.

The ABD matrix is a shell or laminate stiffness representation:

$$\begin{bmatrix} N \\ M \end{bmatrix} = \begin{bmatrix} A & B \\ B & D \end{bmatrix} \begin{bmatrix} \varepsilon_0 \\ \kappa \end{bmatrix}.$$

A is membrane stiffness, B is membrane-bending coupling, and D is bending stiffness. This model is appropriate when the section stiffness has already been homogenized outside FEMaster or when a laminate should be represented directly by engineering stiffness resultants rather than by individual plies.

Example

```
*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000000000.0, 0.3
*DENSITY
7850.0
*THERMALEXPANSION
1.2e-5
```

12 Coordinate Systems and Orientations

*ORIENTATION defines named local coordinate systems. These names can be referenced by sections, loads, supports, and connectors. The purpose is always the same: values that are convenient or physically meaningful in a local basis are transformed into the global basis used by the assembled finite-element equations.

Let the local basis vectors be the columns of a rotation matrix

$$R = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}.$$

A local vector v_ℓ is transformed to global coordinates by

$$v_g = R v_\ell.$$

For displacements, forces, rotations, and moments, the same directional transformation is used for the three-component translational block and the three-component rotational block.

12.1 Rectangular systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=RECTANGULAR
<x1>, <x2>, <x3>, <y1>, <y2>, <y3>, <z1>, <z2>, <z3>
```

RECTANGULAR describes a Cartesian local basis. The command accepts three practical input forms:

- 9 values: three direction vectors are supplied.
- 6 values: two direction vectors are supplied and the third is completed.
- 3 values: one primary direction is supplied and the remaining directions are completed.

The safest input is two or three non-collinear vectors. FEMaster normalizes and completes the basis, but it cannot infer a robust local frame from ambiguous geometry. If two supplied vectors are almost parallel, the cross product becomes small and the resulting local axes may be unstable. In formula form, a typical completion from two vectors a and b is

$$e_1 = \frac{a}{\|a\|}, \qquad e_3 = \frac{e_1 \times b}{\|e_1 \times b\|}, \qquad e_2 = e_3 \times e_1.$$

The exact input variant decides which vector is treated as primary, but the modelling implication is the same: provide directions that define a clear right-handed frame.

12.2 Cylindrical systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=CYLINDRICAL
<origin-x>, <origin-y>, <origin-z>,
<axis-x>, <axis-y>, <axis-z>,
<reference-x>, <reference-y>, <reference-z>
```

`CYLINDRICAL` defines a coordinate system from an origin, an axis direction, and a reference direction. The axial unit vector is

$$e_z = \frac{a}{\|a\|}.$$

At a point x, the radial direction is obtained by projecting $x - o$ onto the plane normal to e_z:

$$r = (x - o) - ((x - o) \cdot e_z)e_z, \qquad e_r = \frac{r}{\|r\|}.$$

The tangential direction follows from

$$e_\theta = e_z \times e_r.$$

Near the cylindrical axis, the radial direction is not well defined. The reference vector is used to establish a stable angular reference, but users should still avoid applying cylindrical local directions exactly on the axis unless the intended direction is unambiguous.

12.3 Where orientations are used

Command	Use of the orientation
`*SUPPORT`	Prescribed support components are interpreted in the local basis before they become constraint equations.
`*CLOAD`	Nodal force and moment components are entered locally and transformed to global generalized forces.
`*DLOAD`	Surface traction components are interpreted in the local basis.
`*VLOAD`	Body-force components are interpreted in the local basis.
`*SOLIDSECTION`	Optional material axes for solid elements.
`*SHELLSECTION`	Optional shell material/resultant axes.
`*CONNECTOR`	Local connector axes define which relative translations or rotations are constrained.

Near-collinear directions

Coordinate systems, beam sections, couplings, and connectors are sensitive to near-collinear direction definitions. A nearly zero cross product creates poorly defined axes and can produce unintuitive constraints or stiffness directions. Prefer clear, normalized, geometrically separated reference directions.

13 Fields

*FIELD defines generic tabular data. Fields may be defined at root level or inside a loadcase block, depending on their intended use.

Syntax

```
*FIELD, NAME=<name>, TYPE=NODE|ELEMENT|ELEMENT_IP, COLS=<n>, FILL=ZERO|NAN
<id>, <v1>, <v2>, ...
```

COLS is required and must be greater than zero. FEMaster limits fields to 64 components. FILL defaults to ZERO; NAN initializes unspecified values with NaN.

Domain	Meaning
NODE	One row per node ID. Used for temperature fields, initial velocities, or prescribed nodal data.
ELEMENT	One row per element ID. Used for topology density and orientation fields.
ELEMENT_IP	Integration-point field. Used for data that is tied to element integration points.

Data lines may omit individual values; omitted values do not overwrite existing entries. The tokens NAN, INF, +INF, and -INF are accepted.

13.1 Typical references

Use	Domain/Cols	Referencing command
Temperature load	NODE, 1	*TLOAD with TEMPERATUREFIELD.
Initial velocity	NODE, 6	*INITIALVELOCITY.
Topology density	ELEMENT, 1	*TOPODENSITY.
Topology orientation	ELEMENT, 3	*TOPOORIENT.

14 Loads, Supports, and Amplitudes

Loads and supports are not activated directly by their definition command. They are assigned to named collectors first. A support collector is a named list of support entries. A load collector is a named list of load entries. A loadcase later selects one or more of these collectors with `*SUPPORTS` and `*LOADS`.

This collector model is important for reusable decks. For example, one support collector may contain all fixture supports, another may contain symmetry supports, and a third may contain temporary stabilization supports. A static loadcase can activate only the fixture collector, while a modal loadcase can activate fixture and symmetry collectors. Loads work the same way: each `*CLOAD`, `*DLOAD`, `*PLOAD`, `*VLOAD`, `*TLOAD`, or `*INERTIALOAD` entry declares which `LOAD_COLLECTOR` it belongs to; the loadcase decides which collectors are assembled.

14.1 Support collectors

Syntax

```
*SUPPORT, SUPPORT_COLLECTOR=<name>, ORIENTATION=<orientation>
<node-set-or-id>, <ux>, <uy>, <uz>, <rx>, <ry>, <rz>
```

`*SUPPORT` adds one support entry to the collector named by `SUPPORT_COLLECTOR`. The collector is created implicitly when the first support uses its name. The data line targets either a node set or a single node id and then gives prescribed values for the six nodal components `Ux, Uy, Uz, Rx, Ry, Rz`. In most models the prescribed values are zero, but nonzero entries can be used for enforced displacement or rotation.

If `ORIENTATION` is omitted, the six values are interpreted in the global coordinate system. If an orientation is given, the translational and rotational components are interpreted in that local coordinate system and transformed into the global system before the constraint equations are built. This is the usual way to describe supports normal to a cylindrical surface, sliding conditions in a skew system, or fixtures that are easier to express in local axes than in global axes.

The support value is a kinematic prescription. In matrix form, a scalar support can be read as

$$e_i^T u = \bar{u}_i,$$

where e_i selects the constrained degree of freedom and $\bar{u}_i$ is the prescribed value from the support line. Multiple support entries in the same collector are assembled into one set of equations.

14.2 Concentrated loads

Syntax

```
*CLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<node-set-or-id>, <Fx>, <Fy>, <Fz>, <Mx>, <My>, <Mz>
```

`*CLOAD` adds nodal forces and moments to the collector named by `LOAD_COLLECTOR`. The target may be a node set or a single node id. The six numeric values correspond to nodal generalized force components conjugate to `Ux, Uy, Uz, Rx, Ry, Rz`. Translational components are forces; rotational components are moments.

If an orientation is specified, the force and moment components are interpreted in the local system and transformed to global components during assembly. If an amplitude is specified, the load entry is multiplied by the amplitude value in transient analyses:

$$f_j(t) = a(t) f_j.$$

In static analyses, the load is used as its defined value unless the loadcase explicitly evaluates time-dependent data.

14.3 Surface traction and pressure

Syntax

```
*DLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<surface-set-or-id>, <Fx>, <Fy>, <Fz>
```

Syntax

```
*PLOAD, LOAD_COLLECTOR=<name>, AMPLITUDE=<amp>
<surface-set-or-id>, <pressure>
```

`*DLOAD` defines a vector traction on surfaces. Conceptually, the load contribution is

$$f_e = \int_{\Gamma_e} N^T t \, d\Gamma,$$

where t is the traction vector and N contains the element shape functions on the loaded surface. `*PLOAD` defines a scalar pressure. A pressure is converted into a traction using the surface normal, so the selected surface side and its normal direction determine the sign convention:

$$t = pn.$$

Use `*DLOAD` when the traction direction is known as a vector. Use `*PLOAD` when the load should follow the surface normal. Both commands add entries to a load collector and are activated only when that collector is referenced by a loadcase.

14.4 Volume loads

Syntax

```
*VLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<element-set-or-id>, <Fx>, <Fy>, <Fz>
```

`*VLOAD` defines a body-force density over an element region. The assembled contribution follows

$$f_e = \int_{\Omega_e} N^T b \, d\Omega,$$

where b is the supplied volume load vector. This is appropriate for load densities that are already expressed per unit volume. If a gravitational acceleration should be converted into force, the material density and the intended unit convention must be reflected in the supplied values or in the load formulation used for the model.

14.5 Thermal loads

Syntax

```
*TLOAD, LOAD_COLLECTOR=<name>, TEMPERATUREFIELD=<field>, REFERENCETEMPERATURE=<T0>
```

`*TLOAD` adds thermal strain loading to a load collector. The command references a nodal temperature field and a reference temperature. For a material with thermal expansion coefficient α, the thermal strain is based on

$$\varepsilon_{\mathrm{th}} = \alpha(T - T_0)\begin{bmatrix}1 & 1 & 1 & 0 & 0 & 0\end{bmatrix}^T$$

for isotropic expansion. The corresponding equivalent mechanical load is assembled from the element constitutive law and the thermal strain. Thermal loads therefore require both a temperature field and material definitions with thermal expansion data.

14.6 Inertial loads

Syntax

```
*INERTIALOAD, LOAD_COLLECTOR=<name>, CONSIDER_POINT_MASSES=0|1
<elset>, <cx>, <cy>, <cz>, <a0x>, <a0y>, <a0z>, <wx>, <wy>, <wz>, <alphax>, <alphay>, <alphaz
    >
```

`*INERTIALOAD` creates equivalent nodal loads from a prescribed rigid-body acceleration field. The first vector c is the reference point. The second vector a_0 is the translational acceleration of that point. The third vector ω is angular velocity and the fourth vector α is angular acceleration. At a material point x, the rigid-body acceleration is

$$a(x) = a_0 + \alpha \times (x - c) + \omega \times (\omega \times (x - c)) .$$

The equivalent inertial load is assembled over the selected element set. `CONSIDER_POINT_MASSES` controls whether point-mass features are included as additional concentrated inertia contributions.

14.7 Amplitudes

Syntax

```
*AMPLITUDE, NAME=<name>, TYPE=LINEAR|STEP|NEAREST
<time>, <value>
```

An amplitude is a scalar time function. Load entries that reference an amplitude are scaled by its value during transient analysis. `LINEAR` interpolates linearly between data points. `STEP` keeps the previous value until the next point is reached. `NEAREST` uses the value of the closest tabulated time. Amplitudes are useful when the spatial load pattern is fixed but the intensity changes over time.

15 Constraints

Constraints define kinematic equations between degrees of freedom. They are different from supports in modelling intent: a support prescribes motion of selected degrees of freedom, while a constraint relates several nodes, surfaces, or reference points to each other. FEMaster assembles supports and constraints into one algebraic equation set before the active loadcase is solved.

Most constraint equations can be written as

$$Cu = d.$$

u is the vector of active nodal degrees of freedom, C is the constraint matrix, and d is the right-hand side. Homogeneous constraints have $d = 0$. The constraint definitions in this chapter describe which equations are generated. The numerical handling of those equations, for example Nullspace reduction or Lagrange multipliers, is selected by `*CONSTRAINTMETHOD` and described in Chapter 16.

15.1 Rigid-body motion suppression: `*RBM`

Syntax

```
*RBM, ELSET=<elset>
```

`*RBM` generates up to six homogeneous equations for the nodes used by the selected element set. `ELSET` also accepts the alias `SET`; if omitted, the default is `EALL`. The command is meant to suppress free rigid-body modes without fixing an arbitrary node. This is especially useful for stabilization, diagnostic models, and inertia-relief workflows.

FEMaster first collects all nodes that belong to the selected structural elements. It computes the center of gravity x_0 of the selected element region and uses the relative position

$$r_i = x_i - x_0$$

for every participating node i. With n participating nodes and $w = 1/n$, the three translational equations are

$$\sum_i w u_{ix} = 0, \qquad \sum_i w u_{iy} = 0, \qquad \sum_i w u_{iz} = 0.$$

The three rotational equations suppress the average rigid rotation measured by the first moment of the displacement field:

$$\sum_i w (r_i \times u_i)_x = 0, \qquad \sum_i w (r_i \times u_i)_y = 0, \qquad \sum_i w (r_i \times u_i)_z = 0.$$

Expanded component-wise, these are

$$\sum_i w (r_{iy} u_{iz} - r_{iz} u_{iy}) = 0,$$

$$\sum_i w(r_{iz}u_{ix} - r_{ix}u_{iz}) = 0,$$

$$\sum_i w(r_{ix}u_{iy} - r_{iy}u_{ix}) = 0.$$

Only active translational degrees of freedom are used. If a required component does not exist for a node, it is skipped for that equation.

15.2 Coupling: *COUPLING

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SLAVE=<slave-nset>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SFSET=<surface-set>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

*COUPLING connects one master node set to either a slave node set or a slave surface set. Exactly one of SLAVE and SFSET must be given. In practical models the master set should normally contain one reference node. Larger master sets can over-constrain the model because the generated equations are based on a single master node.

The data line contains six flags for Ux, Uy, Uz, Rx, Ry, Rz. Values greater than zero activate the corresponding coupling component. KINEMATIC generates constraint equations. STRUCTURAL does not generate kinematic equations; it distributes generalized loads from the master node to slave nodes in a work-equivalent way.

15.2.1 Kinematic coupling equations

For a slave node s, master node m, and offset

$$r = x_s - x_m,$$

the translational kinematic relation is the small-rotation rigid-body relation

$$u_s = u_m + \theta_m \times r.$$

FEMaster writes this component-wise as homogeneous equations. For example,

$$u_{sx} - u_{mx} - r_z\theta_{my} + r_y\theta_{mz} = 0,$$

$$u_{sy} - u_{my} - r_x\theta_{mz} + r_z\theta_{mx} = 0,$$

$$u_{sz} - u_{mz} - r_y\theta_{mx} + r_x\theta_{my} = 0.$$

Rotational coupling is direct:

$$\theta_{sx} - \theta_{mx} = 0, \qquad \theta_{sy} - \theta_{my} = 0, \qquad \theta_{sz} - \theta_{mz} = 0.$$

Only the components enabled by the six flags are generated, and only if the involved degrees of freedom exist.

15.2.2 Structural coupling load distribution

For `STRUCTURAL`, a generalized master load

$$b = \begin{bmatrix} F_x & F_y & F_z & M_x & M_y & M_z \end{bmatrix}^T$$

is distributed to slave translational forces. FEMaster builds the equilibrium operator

$$A = \begin{bmatrix} I & I & \dots & I \\ [r_1]_\times & [r_2]_\times & \dots & [r_n]_\times \end{bmatrix},$$

where $[r_i]_\times f_i = r_i \times f_i$. The distributed force vector is computed in a least-norm, work-equivalent form from

$$G\lambda = b, \qquad G = AA^T, \qquad f = A^T\lambda.$$

The resulting slave forces reproduce the master resultant force and moment as closely as the slave geometry permits. The master load is then consumed because it has been represented by slave forces.

15.3 Tie: `*TIE`

Syntax

```
*TIE, MASTER=<master>, SLAVE=<slave>, ADJUST=NO|YES, DISTANCE=<search-distance>
```

`*TIE` binds slave nodes to a master surface set or master line set. The slave region may be a node set or a surface set. For each slave node, FEMaster searches the closest master geometry within `DISTANCE`. If no candidate is found, no equation is generated for that slave node. If `ADJUST=YES`, the slave node coordinate is moved to the projected master position before the equations are assembled.

Let s be a slave node projected onto a master surface or line at local coordinates ξ. Let $N_a(\xi)$ be the shape function of master node a at that projected point. For every active degree of freedom, the compatibility equation is

$$u_s - \sum_a N_a(\xi) u_a = 0.$$

This ties the slave degree of freedom to the interpolated master motion. For surface masters the interpolation uses the surface shape functions; for line masters it uses the line shape functions. The same equation form is used for translational and rotational degrees of freedom when those degrees of freedom exist on both slave and all master nodes.

15.4 Connector: `*CONNECTOR`

Syntax

```
*CONNECTOR, TYPE=<type>, NSET1=<nset1>, NSET2=<nset2>, COORDINATESYSTEM=<orientation>
```

`*CONNECTOR` creates idealized relative-motion constraints between two node sets in a named coordinate system. Supported types are `BEAM`, `HINGE`, `CYLINDRICAL`, `TRANSLATOR`, `JOIN`, and `JOINRX`. The connector type maps to a six-component mask that decides which local translations and rotations are constrained.

For each constrained local component e_α, FEMaster transforms the local direction into the global system:

$$g_\alpha = Re_\alpha.$$

For a constrained translation, the generated equation is

$$g_\alpha^T(u_1 - u_2) = 0.$$

For a constrained rotation, the equation is

$$g_\alpha^T(\theta_1 - \theta_2) = 0.$$

The connector therefore constrains relative motion along selected local axes rather than simply tying global components. This is why the referenced coordinate system is part of the connector definition and should be chosen carefully.

15.5 Constraint summary

Syntax

```
*LOADCASE, TYPE=LINEARSTATIC
  *CONSTRAINTSUMMARY
```

`*CONSTRAINTSUMMARY` requests a diagnostic listing of the internally generated constraint equations for the active loadcase. The output is grouped by constraint origin, for example supports, RBM equations, couplings, ties, and connectors. This is useful when a model is unexpectedly singular, over-constrained, or when a surface tie did not find as many slave projections as intended. It is a diagnostic report of generated equations, not a modelling command that changes the equations.

Master sets

Many coupling and connector workflows are intended for one reference node in the master set. Larger master sets may over-constrain the model or produce a kinematic interpretation that is different from the intended reference-point behaviour.

16 Analysis and Solver Controls

This chapter describes commands that are used inside loadcases to activate collectors, choose the linear solver, select the constraint backend, and configure analysis-specific numerical options. Constraint definitions themselves are described in Chapter 15; the commands here decide how those definitions are used in a particular analysis.

16.1 Activating support and load collectors

Syntax

```
*SUPPORTS
<support-collector-1>, <support-collector-2>, ...

*LOADS
<load-collector-1>, <load-collector-2>, ...
```

`*SUPPORTS` activates named support collectors that were populated by `*SUPPORT`. `*LOADS` activates named load collectors that were populated by `*CLOAD`, `*DLOAD`, `*PLOAD`, `*VLOAD`, `*TLOAD`, or `*INERTIALOAD`. The loadcase does not activate every support or load in the model automatically. It assembles only the collectors listed in these commands.

Mathematically, active support collectors contribute equations to the global constraint set $Cu = d$. Active load collectors contribute to the right-hand side vector f. In a transient loadcase, time-dependent amplitudes are evaluated during time integration, so the active load vector is a function $f(t)$.

16.2 Solver

Syntax

```
*SOLVER, DEVICE=CPU|GPU, METHOD=DIRECT|INDIRECT
```

`DEVICE` selects the execution device. `CPU` is the standard path. `GPU` is intended for configurations where the corresponding solver routines and data structures are available. `METHOD` selects the strategy for linear systems.

`DIRECT` solves a system by factorization. For a static reduced system

$$Aq = b,$$

the matrix A is factorized and the solution is obtained by triangular solves or an equivalent direct procedure. Direct solvers are robust and predictable for small and medium models. Their main limitation is memory growth, especially for large three-dimensional meshes.

`INDIRECT` uses an iterative solution path. Instead of building a full direct factorization, an approximate solution is improved until the residual is sufficiently small. Iterative methods can be attractive for large systems because they avoid some of the memory cost of direct factorization. They are more sensitive to conditioning, scaling, constraint handling, and matrix definiteness. In FEMaster the indirect path should be used with `NULLSPACE` constraints, because the reduced system keeps the standard structural form. It is not intended for the saddle-point system produced by Lagrange multipliers.

Constraint	Backend	DIRECT	INDIRECT
`NULLSPACE`	CPU MKL	Yes	Yes
`NULLSPACE`	CPU Eigen	Yes	Yes
`NULLSPACE`	GPU	Yes	Yes
`NULLSPACE`	GPU cuDSS	Yes	Yes
`LAGRANGE`	CPU MKL	Yes	No
`LAGRANGE`	CPU Eigen	Limited	No
`LAGRANGE`	GPU	No	No
`LAGRANGE`	GPU cuDSS	Yes	No

Table 16.1: Supported static solver and constraint combinations.

16.3 Constraint method

Syntax

```
*CONSTRAINTMETHOD, TYPE=NULLSPACE|LAGRANGE
```

The active supports and constraints define

$$Cu = d.$$

`*CONSTRAINTMETHOD` selects how this equation set is introduced into the analysis equations.

16.3.1 Nullspace method

`NULLSPACE` parameterizes all admissible displacements as

$$u = u_p + Tq.$$

u_p is a particular displacement vector satisfying the inhomogeneous part of the constraints. T is a basis for the nullspace of C, so

$$CT = 0, \qquad Cu_p = d.$$

q contains only independent degrees of freedom. A static system

$$Ku = f$$

is transformed into

$$T^T K T q = T^T (f - K u_p).$$

The matrix

$$A = T^T K T$$

is the reduced stiffness matrix. This approach removes constrained degrees of freedom from the solve and is well suited for direct and iterative solvers. It also avoids adding Lagrange multipliers to the unknown vector.

For eigenvalue and transient problems the same projection is applied to the operators:

$$K_r = T^T K T, \qquad M_r = T^T M T, \qquad C_r = T^T C T.$$

Recovered displacements are mapped back with $u = u_p + Tq$, while velocities and accelerations use the homogeneous part $T\dot{q}$ and $T\ddot{q}$.

16.3.2 Lagrange method

`LAGRANGE` keeps the original displacement vector and adds one multiplier per constraint equation. The static system becomes

$$\begin{bmatrix} K & C^T \\ C & -\epsilon I \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}.$$

λ contains the Lagrange multipliers, which can be interpreted as generalized constraint reactions. ϵ is a possible small regularization term when configured by the solver path; conceptually, the pure Lagrange system has $\epsilon = 0$.

This formulation is direct and preserves the original displacement variables, but it creates a saddle-point problem. Such systems are indefinite and more demanding numerically. In FEMaster, `LAGRANGE` requires the direct linear solver for linear static analysis; see Table 16.1 for supported backend combinations. Use `NULLSPACE` when an indirect solver is desired.

16.4 Inertia relief

Syntax

```
*INERTIARELIEF, CONSIDER_POINT_MASSES=0|1
```

Inertia relief is used for static analysis of free or nearly free structures. A free body under an unbalanced external load has no static equilibrium in an inertial frame; it accelerates as a rigid body. Inertia relief computes an equivalent rigid-body acceleration and adds inertia loads that balance the external resultant force and moment. The remaining static solve describes elastic deformation relative to that accelerating rigid-body motion.

This is not a substitute for missing real supports. If the physical model has bearings, bolts, fixtures, or symmetry conditions, they should be modelled as supports or constraints. Inertia relief is appropriate when the analysed body is intentionally free, for example an aircraft component under manoeuvre load or a structure in a load case where fixed ground supports would be artificial.

`CONSIDER_POINT_MASSES` controls whether point-mass features are included in the mass properties used by inertia relief. If point masses represent relevant equipment or lumped mass, set this option to 1. If they are purely numerical or should not affect the inertial balance, leave them out.

16.5 Load rebalancing

Syntax

```
*REBALANCELOADS
```

Load rebalancing modifies the external load balance so that the resultant force and moment vanish. It is a numerical balancing tool, not a physical rigid-body acceleration model. It is useful when a load is intended to be self-equilibrated but small residual resultants appear because of discretization, surface selection, rounding, or simplified load introduction.

The distinction from inertia relief is important. Inertia relief says: the structure is free, the external load would accelerate it, and inertia loads should represent that acceleration. Load rebalancing says: the load

should be balanced, and the residual imbalance should be corrected. For real free-body operating loads, inertia relief is usually the more physical interpretation. For small unintended imbalance in an otherwise self-equilibrated load, rebalancing is often the simpler diagnostic option.

16.6 Matrix and diagnostic requests

Syntax

```
*REQUESTSTIFFNESS, FILE=<base-file>
*REQUESTSTGEOM, FILE=<base-file>
*CONSTRAINTSUMMARY
```

`*REQUESTSTIFFNESS` requests stiffness matrix output for diagnostics or external comparison. `*REQUESTSTGEOM` requests geometric stiffness output in buckling analysis. `*CONSTRAINTSUMMARY` prints the internally generated constraint equations grouped by origin, such as supports, RBM equations, couplings, ties, and connectors.

These commands should be treated as diagnostic output controls. They do not change the physical model. They are helpful when validating assemblies, comparing reduced matrices, debugging constraint generation, or checking why a loadcase is singular or over-constrained.

16.7 Eigenvalue parameters

Syntax

```
*NUMEIGENVALUES
<count>

*SIGMA
<shift>
```

`*NUMEIGENVALUES` sets how many eigenpairs are requested for eigenfrequency or buckling analysis. The value must be positive.

`*SIGMA` is used in linear buckling. The reduced buckling problem is assembled as

$$A\phi = \lambda(-B)\phi,$$

where

$$A = T^T K T, \qquad B = T^T K_G T.$$

The reported buckling factors are sorted ascending after the eigensolve. The shift σ guides the buckling eigenvalue search and is the smallest buckling factor region that should be targeted for output. It must be chosen carefully: a poor shift can make the eigensolver converge to an unintended part of the spectrum or miss the factors of engineering interest.

If `*SIGMA` is set to 0, FEMaster generates a shift automatically. It first tries a Rayleigh estimate from the pre-buckling displacement q_{pre}:

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T (-B) q_{\text{pre}}}.$$

If that estimate is finite and positive, the generated shift is

$$\sigma = \frac{\lambda_{\text{raw}}}{10^6}.$$

If the Rayleigh estimate is not usable, FEMaster falls back to a diagonal ratio estimate

$$\lambda_i = \frac{A_{ii}}{(-B)_{ii}}$$

for positive finite diagonal entries and uses a conservative lower quartile of those values. If no valid estimate exists, the fallback shift is 10^{-12}.

16.8 Transient-specific controls

Syntax

```
*TIME
<t_start>, <t_end>, <dt>
```

`*TIME` defines the time interval and fixed time step for `LINEARTRANSIENT`. It accepts either three values $(t_{\text{start}}, t_{\text{end}}, \Delta t)$ or two values $(t_{\text{end}}, \Delta t)$, in which case the start time is 0. The number of integration steps is based on the interval length and Δt. A smaller time step gives better temporal resolution but increases the number of linear solves and result states.

Syntax

```
*NEWMARK
<beta>, <gamma>
```

`*NEWMARK` sets the Newmark integration parameters β_N and γ_N. The transient equation is

$$M\ddot{u} + C\dot{u} + Ku = f(t).$$

In reduced coordinates FEMaster solves the projected form

$$M_r\ddot{q} + C_r\dot{q} + K_r q = r(t).$$

The common values $\beta_N = 0.25$ and $\gamma_N = 0.5$ correspond to the average-acceleration method for linear systems. Other values should be chosen deliberately because they change numerical damping, stability, and accuracy.

Syntax

```
*DAMPING, TYPE=RAYLEIGH
<alpha>, <beta>
```

`*DAMPING` currently supports Rayleigh damping:

$$C = \alpha M + \beta K.$$

α controls mass-proportional damping and has stronger influence at low frequencies. β controls stiffness-proportional damping and has stronger influence at high frequencies. In reduced form FEMaster builds

$$C_r = \alpha M_r + \beta K_r.$$

Syntax

```
*WRITEEVERY, TYPE=STEPS|TIME
<value>
```

`*WRITEEVERY` controls transient result cadence. With `TYPE=STEPS`, the value is rounded to an integer and results are written every N steps. With `TYPE=TIME`, the value is a physical output interval; FEMaster converts it to a step stride by dividing by the analysis time step. The final time state is written even when it does not fall exactly on the stride.

Syntax

```
*INITIALVELOCITY, FIELD=<node-field>
```

`*INITIALVELOCITY` references a node field with six components. The field is reduced into the active coordinate space and used as the initial velocity. If no initial velocity is specified, the initial reduced velocity is zero. Initial acceleration is computed internally from the initial state, the active matrices, and the load at the start time.

17 Loadcases

A loadcase defines which analysis is executed on the previously defined model. The model describes what exists: nodes, elements, surfaces, sets, materials, sections, features, loads, supports, constraints, and fields. The loadcase describes which equation is assembled, which collectors are active, how constraints are handled, which solver is used, and which analysis-specific parameters are applied.

17.1 General syntax

Syntax

```
*LOADCASE, TYPE=<type>, NAME=<optional-name>
  <loadcase commands>
```

`TYPE` is required. `NAME` is optional and is used for readability in the input deck and result output. Loadcases are executed in input order. Result blocks are written in the same order.

17.2 Common equation structure

Linear loadcases use global matrices assembled from element contributions. The most common operators are stiffness K, mass M, damping C, and geometric stiffness K_G. Active load collectors produce a right-hand side f. Active supports and constraints produce equations

$$C_c u = d.$$

The selected constraint method then either reduces the problem with $u = u_p + Tq$ or augments it with Lagrange multipliers. The loadcase formulas below are written in their physical full-space form first; the actual solved system may be the reduced or augmented form described in Chapter 16.

17.3 Linear Static

`LINEARSTATIC` solves small-displacement linear static equilibrium. It is the standard analysis for displacements, reactions, strains, stresses, and static load paths.

17.3.1 Governing equation

The physical equation is

$$Ku = f.$$

After Nullspace constraint reduction, FEMaster solves

$$T^T K T q = T^T (f - K u_p), \qquad u = u_p + Tq.$$

With Lagrange constraints, the static system is

$$\begin{bmatrix} K & C_c^T \\ C_c & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}$$

up to possible numerical regularization.

17.3.2 Typical DSL

Example

```
*LOADCASE, TYPE=LINEARSTATIC, NAME=static-load
  *SUPPORTS
  FIXED
  *LOADS
  FORCE
  *SOLVER, DEVICE=CPU, METHOD=DIRECT
  *CONSTRAINTMETHOD, TYPE=NULLSPACE
```

Common commands are `*SUPPORTS`, `*LOADS`, `*SOLVER`, and `*CONSTRAINTMETHOD`. Optional controls include `*INERTIARELIEF`, `*REBALANCELOADS`, `*REQUESTSTIFFNESS`, and `*CONSTRAINTSUMMARY`.

17.4 Linear Static Topology

`LINEARSTATICTOPO` solves a static problem whose element stiffness can be modified by topology fields. It is useful for topology studies, density-based parameter sweeps, and orientation-dependent stiffness experiments.

17.4.1 Governing equation

The equation is

$$K(\rho, \theta)u = f.$$

ρ is an element density-like field and θ is an element orientation-like field. The density field is typically used through a penalized stiffness contribution

$$K_e^{\text{eff}} = \rho_e^p K_e,$$

where p is supplied by `*TOPOEXPONENT`. Low densities reduce stiffness contribution; high densities approach the full element stiffness. The orientation field is used by element/material formulations that evaluate local orientation data.

17.4.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARSTATICTOPO, NAME=<name>
  *TOPODENSITY
  <element-field>
  *TOPOORIENT
  <element-field>
  *TOPOEXPONENT
  <p>
```

`*TOPODENSITY` requires an element field with one component. `*TOPOORIENT` requires an element field with three components. The static controls from `LINEARSTATIC` are used in the same way.

The compliance, density sensitivity, and orientation sensitivity derivations are collected in Appendix A.

17.5 Eigenfrequency

`EIGENFREQ` computes natural frequencies and mode shapes of the linearized model. Loads are not used because the free-vibration problem is defined by stiffness, mass, and boundary conditions.

17.5.1 Governing equation

The full-space eigenproblem is

$$K\phi_i = \omega_i^2 M\phi_i.$$

ϕ_i is the mode shape and ω_i is the circular frequency. The frequency in Hertz is

$$f_i = \frac{\omega_i}{2\pi}.$$

With Nullspace constraints, the reduced problem is

$$T^T KT\psi_i = \omega_i^2 T^T MT\psi_i, \qquad \phi_i = T\psi_i.$$

17.5.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=EIGENFREQ, NAME=<name>
  *SUPPORTS
  <support-collector>
  *NUMEIGENVALUES
  <count>
```

A free-free model can contain rigid-body modes with zero or near-zero frequency. A supported model computes modes relative to the active supports and constraints.

17.6 Linear Buckling

`LINEARBUCKLING` computes ideal linear buckling factors and buckling modes around a preloaded reference state. It first solves a static preload problem, then assembles geometric stiffness from the resulting stresses, and finally solves a generalized eigenvalue problem.

17.6.1 Governing equations

The preload solve is

$$Ku_0 = f_{\text{pre}}.$$

With constraints, FEMaster solves the reduced preload equation

$$Aq_{\text{pre}} = b, \qquad A = T^T KT, \qquad b = T^T(f_{\text{pre}} - Ku_p).$$

From the preload stresses, FEMaster builds the geometric stiffness K_G and reduces it to

$$B = T^T K_G T.$$

The buckling eigenproblem is solved as

$$A\phi_i = \lambda_i(-B)\phi_i.$$

λ_i is the buckling factor. For the preload pattern f_{pre}, the ideal linear critical load for mode i is

$$f_{\text{crit},i} = \lambda_i f_{\text{pre}}.$$

17.6.2 Sigma

`*SIGMA` controls the buckling eigenvalue shift. It should be set carefully because it guides the part of the spectrum returned by the eigensolver. If `*SIGMA` is zero, FEMaster estimates a shift automatically from the preload displacement with the Rayleigh quotient

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T(-B)q_{\text{pre}}}.$$

When this estimate is valid and positive, the generated shift is $\lambda_{\text{raw}}/10^6$. Otherwise a diagonal-ratio fallback is used; if that also fails, 10^{-12} is used.

17.6.3 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARBUCKLING, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <preload-collector>
  *NUMEIGENVALUES
  <count>
  *SIGMA
  <shift>
```

`*LOADS` defines the preload. `*REQUESTSTIFFNESS` and `*REQUESTSTGEOM` can be used to write elastic and geometric stiffness matrices for diagnostics.

17.7 Linear Transient

`LINEARTRANSIENT` solves the linear dynamic response over time with implicit Newmark integration. The structure remains linear; contact, plasticity, and large deformation effects are not included in this loadcase.

17.7.1 Governing equation

The physical equation is

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = f(t).$$

With Nullspace constraints, this becomes

$$M_r\ddot{q}(t) + C_r\dot{q}(t) + K_r q(t) = r(t),$$

with

$$M_r = T^T M T, \qquad C_r = T^T C T, \qquad K_r = T^T K T.$$

Loads with amplitudes are assembled as

$$f(t) = \sum_j a_j(t) f_j.$$

For Rayleigh damping,

$$C = \alpha M + \beta K.$$

17.7.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARTRANSIENT, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <load-collector>
  *TIME
  <t_start>, <t_end>, <dt>
  *NEWMARK
  <beta>, <gamma>
  *DAMPING, TYPE=RAYLEIGH
  <alpha>, <beta>
  *WRITEEVERY, TYPE=STEPS
  <n>
```

The transient-specific keywords *TIME, *NEWMARK, *DAMPING, *WRITEEVERY, and *INITIALVELOCITY are described in Chapter 16.

17.8 Typical command overview

Loadcase	Typical commands
LINEARSTATIC	*SUPPORTS, *LOADS, *SOLVER, *CONSTRAINTMETHOD, optional *INERTIARELIEF, *REBALANCELOADS, *REQUESTSTIFFNESS, *CONSTRAINTSUMMARY.
LINEARSTATICTOPO	Linear-static commands plus *TOPODENSITY, *TOPOORIENT, *TOPOEXPONENT.
EIGENFREQ	*SUPPORTS, *CONSTRAINTMETHOD, *NUMEIGENVALUES.
LINEARBUCKLING	*SUPPORTS, *LOADS, *CONSTRAINTMETHOD, *NUMEIGENVALUES, optional *SIGMA, *REQUESTSTIFFNESS, *REQUESTSTGEOM.
LINEARTRANSIENT	*SUPPORTS, *LOADS, *SOLVER, *CONSTRAINTMETHOD, *TIME, *NEWMARK, *DAMPING, *WRITEEVERY, *INITIALVELOCITY.

18 Result Format

FEMaster writes text-based `.res` files. The file is organized into loadcases and fields. Each loadcase starts with a loadcase header followed by any number of field blocks.

18.1 Loadcase blocks

Syntax

```
LOADCASE <id> <name-or-type>
```

The loadcase ID follows execution order. The following field blocks belong to that loadcase until the next loadcase header appears.

18.2 Dense fields

Syntax

```
FIELD <name> TYPE=<type> ROWS=<rows> COLS=<cols>
<v11> <v12> ...
```

`TYPE` is always written and is one of `NODE`, `ELEMENT`, `ELEMENT_NODAL`, `ELEMENT_IP`, or `UNKNOWN`. `ROWS` gives the number of rows. `COLS` gives the number of values per row. For nodal fields, a row typically corresponds to a node-position row. For element fields, a row corresponds to an element-position row.

18.3 Indexed fields

Syntax

```
FIELD <name> TYPE=<type> ROWS=<rows> INDEX_COLS=<i> VALUE_COLS=<v>
<index...> <value...>
```

Some result quantities are not simple node or element fields. Indexed fields use index columns to identify element, integration point, section point, or local result location, followed by value columns.

18.4 Common field names

Loadcase	**Typical fields**
Linear Static	`DISPLACEMENT`, `STRAIN`, `STRESS`, `EXTERNAL_FORCES`, `REACTION_FORCES`, shell and section fields.
Linear Static Topology	Static fields plus `COMPLIANCE`, `DENS_GRAD`, `VOLUME`, `DENSITY`, and optional orientation fields.
Eigenfrequency	`MODE_SHAPE_i`, `PARTICIPATION_i`, `EIGENVALUES`, `EIGENFREQUENCIES`, `FREQUENCIES`.
Linear Buckling	`BUCKLING_MODE_i`, `BUCKLING_FACTORS`.
Linear Transient	`DISPLACEMENT_k`, `VELOCITY_k`, `ACCELERATION_k` for written time steps.

A six-component nodal motion field uses `Ux, Uy, Uz, Rx, Ry, Rz`. A six-component stress or strain field uses the engineering order x, y, z, yz, zx, xy. These conventions must not be confused.

19 Examples

19.1 Linear static

See Section 17.3 for the governing equation and a minimal static loadcase.

19.2 Eigenfrequency

Example

```
*LOADCASE, TYPE=EIGENFREQ
  *SUPPORTS
  FIX
  *NUMEIGENVALUES
  12
```

19.3 Linear buckling

Example

```
*LOADCASE, TYPE=LINEARBUCKLING
  *SUPPORTS
  FIX
  *LOADS
  PRELOAD
  *NUMEIGENVALUES
  6
  *SIGMA
  0.0
  *REQUESTSTGEOM, FILE=buckling_geom
```

19.4 Transient load with amplitude

Example

```
*AMPLITUDE, NAME=RAMP, TYPE=LINEAR
0.0, 0.0
1.0, 1.0

*CLOAD, LOAD_COLLECTOR=F_RAMP, AMPLITUDE=RAMP
TIP, 0, 0, -1000, 0, 0, 0

*LOADCASE, TYPE=LINEARTRANSIENT
  *SUPPORTS
  FIX
  *LOADS
  F_RAMP
  *TIME
  0.0, 1.0, 0.01
  *NEWMARK
  0.25, 0.5
  *WRITEEVERY, TYPE=STEPS
  10
```

19.5 Point mass

Example

```
*NSET, NAME=MASS_NODE
100

*POINTMASS, NSET=MASS_NODE
5.0, 0.02, 0.02, 0.04, 0, 0, 0, 0, 0, 0
```

A Mathematical Derivations

This appendix collects derivations that are useful for understanding the topology-related quantities written by FEMaster. The main reference chapters describe how to use the DSL commands. This appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B \, d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) \, |\det J(r)| \, d\xi \, d\eta \, d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q, this becomes

$$K_{e,0} \approx \sum_{q=1}^{Q} w_q B(r_q)^T D B(r_q) \, |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T K u.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention should therefore be interpreted together with the optimizer interface and the field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \\ 0 & \sin\alpha_1 & \cos\alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos\alpha_2 & 0 & \sin\alpha_2 \\ 0 & 1 & 0 \\ -\sin\alpha_2 & 0 & \cos\alpha_2 \end{bmatrix}, \qquad R_3 = \begin{bmatrix} \cos\alpha_3 & -\sin\alpha_3 & 0 \\ \sin\alpha_3 & \cos\alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & 2\varepsilon_{yz} & 2\varepsilon_{xz} & 2\varepsilon_{xy} \end{bmatrix}^T,$$

and the stress vector is

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{yz} & \sigma_{xz} & \sigma_{xy} \end{bmatrix}^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R. The rotated constitutive matrix used in global strain coordinates is

$$D_{\mathrm{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\mathrm{rot}}(\alpha_e) B \, d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^{Q} w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \qquad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2\rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\frac{\partial R_1}{\partial \alpha_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin\alpha_1 & -\cos\alpha_1 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \end{bmatrix},$$

$$\frac{\partial R_2}{\partial \alpha_2} = \begin{bmatrix} -\sin\alpha_2 & 0 & \cos\alpha_2 \\ 0 & 0 & 0 \\ -\cos\alpha_2 & 0 & -\sin\alpha_2 \end{bmatrix},$$

$$\frac{\partial R_3}{\partial \alpha_3} = \begin{bmatrix} -\sin\alpha_3 & -\cos\alpha_3 & 0 \\ \cos\alpha_3 & -\sin\alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$
$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T / \partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab} R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j} \left(R_{ab} R_{cd} \right) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R, $\partial R / \partial \alpha_j$, T, and $\partial T / \partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.